

Seminar Report

submitted in partial fulfillment of the requirements
for the course “Applied Computer Science”

Parallel Seismic Wave Forward Modeling

Maxim Barnstorf & Utkarsh Pathak

Institute of Computer Science

Georg-August-Universität Göttingen

September 30, 2026

Georg-August-Universität Göttingen
Institute of Computer Science

Goldschmidtstraße 7
37077 Göttingen
Germany

 +49 (551) 39-172000

 +49 (551) 39-14403

 office@informatik.uni-goettingen.de

 www.informatik.uni-goettingen.de

First Supervisor: Patrick Höhn
Second Supervisor:

Declaration on the use of AI tools in the context of examinations

In this work we have used AI tools as follows:

- ☐ Not at all
- ☒ During brainstorming
- ☐ When creating the outline
- ☒ To write individual passages, altogether to the extent of 5-10% of the entire text
- ☒ For the development of software source texts
- ☒ For optimizing or restructuring software source texts
- ☒ For proofreading or optimizing

We hereby declare that we have stated all uses completely and truthfully.

Missing or incorrect information will be considered as an attempt to cheat.

We hereby declare that we have written this thesis independently without any help from others and without the use of documents or aids other than those stated. We have mentioned all used sources and cited them correctly according to established academic citation rules.

Göttingen, September 30, 2026

Abstract

Seismic wave forward modeling is a computationally demanding but essential tool in exploration geophysics, underlying both direct simulation studies and iterative methods such as full waveform inversion. This work presents a hybrid MPI and OpenMP implementation for elastic wave propagation using a second order accurate, staggered grid finite difference scheme, applied to the Marmousi2 subsurface model at full resolution. We discuss several parallelization improvements applied to this implementation, including a two dimensional domain decomposition that balances subdomain shape against communication cost, halo exchanges overlapped with interior computation to hide communication latency, cache aware tiling combined with SIMD vectorization to reduce memory traffic between the stress and velocity updates, and parallelized, compressed output writing to avoid serializing I/O through a single process. Compared to a sequential baseline runtime of 12374 seconds, the parallel implementation achieves a runtime of 203 seconds using 960 CPU cores across ten nodes, corresponding to a speedup of 61 times and a parallel efficiency of 6.4%. The results show that most of the achievable gains are captured at moderate core counts, with just 16 cores already achieving a speedup of 6.3 times, while OpenMP synchronization overhead, MPI communication and memory bottlenecks reduce scaling at larger core counts, motivating GPU acceleration as a promising direction for future work.

Contents

1	Introduction	1
2	Background	2
2.1	Marmousi2 Model	2
2.2	Elastodynamic wave equation	2
2.3	Ricker Wavelet	3
2.4	Numerical Stability and the CFL Condition	4
2.5	Absorbing Boundary Damping	4
2.6	Staggering	4
2.6.1	Second order accuracy	5
2.6.2	Spatial Staggering	5
2.6.3	Temporal Staggering	6
2.7	Tiling	6
3	Methodology	7
3.1	Output	7
3.2	Sequential Design	7
3.2.1	Structure of the program	7
3.2.2	The Division Between Python and C	8
3.2.3	Staggering	8
3.3	Parallel Design	8
3.3.1	Parallelization Coverage	8
3.3.2	MPI Parallelization	9
3.4	OpenMP Parallelization	12
3.4.1	Tile Division	12
3.4.2	Vectorization with SIMD	13
3.4.3	Handling the Domain Boundary When Tiling	13
3.4.4	Parallelizing the Edge Kernels	13
4	Implementation	14
4.1	Setup	14
4.2	Sequential	14
4.2.1	From Continuous Derivatives to Array Differences	14
4.2.2	The Stress Kernel	14
4.2.3	The Velocity Kernel	15
4.2.4	Tiling the Loop Nest	15
4.3	Parallel	16
4.3.1	MPI	16
4.3.2	OpenMP	17
5	Results	19
5.1	Wave Propagation Data	19
5.2	Sequential Performance	20
5.2.1	Overall Runtime	20
5.2.2	Runtime Breakdown by Program Phase	20

5.3	Parallel Performance	21
5.3.1	Strong Scaling	22
5.3.2	Weak Scaling	24
5.3.3	MPI Ranks and Threads per Rank	26
5.3.4	Non-Tiling Version	27
5.3.5	Kernel Comparison	28
5.3.6	Output Compression	29
5.4	Trace Based Analysis with Vampir	30
5.4.1	Measurement Setup	30
5.4.2	OpenMP Barriers	31
5.4.3	Gzip, LZ4 Comparison	31
6	Discussion	33
6.1	Analysis and Bottleneck	33
6.2	Improvements	33
7	Conclusion	35
A	Bibliography	36
B	Work Sharing	37
B.1	Maxim Barnstorf	37
B.2	Utkarsh Pathak	37

1 Introduction

Seismic wave forward modeling is the numerical simulation of how seismic waves propagate through a given earth model. It is used in applications ranging from earthquake hazard assessment and ground motion prediction to subsurface imaging in exploration geophysics. It is also a core building block of inverse problems such as full waveform inversion and seismic tomography, where synthetic waveforms produced by forward modeling are compared against observed data to iteratively refine an estimate of the subsurface.

Solving the elastodynamic wave equation numerically is most commonly done through discretization schemes such as the finite-difference method, in which the spatial and temporal derivatives of the equations are approximated on a grid. Accurately capturing the frequency content and spatial detail required for realistic wave propagation demands high computational power and, in particular, large amounts of memory. While the computation performed at each grid point is simple, many such points need to be updated over many iterations to satisfy the numerical stability conditions required to produce accurate results.

As a result, researchers and industry participants rely on HPC datacenters and the tools available on them to perform these computations within a reasonable amount of time. In modern HPC, this largely comes down to parallelization frameworks such as OpenMP[1] and MPI[2], which allow a developer to split the workload across multiple cores within a node and across multiple nodes in a cluster, respectively. Using both together makes it possible to scale a simulation well beyond what a single machine could handle, provided the workload is divided and coordinated correctly.

One major application of this kind of modeling that is commonly performed with parallelization is in the oil and gas industry. To characterize the subsurface and locate oil and gas reservoirs, pressure is generated at the surface, and the resulting waves traveling through the ground or water are recorded. From these measurements, an approximate model of the subsurface is constructed. Forward modeling is then run on that model to simulate the resulting wave propagation, and the outcome is compared against the actual measurements to assess how closely the model matches reality.

To ground this in a concrete case, the numerical scheme and parallelization strategy examined in this work are applied to the Marmousi2[3] model, a widely used subsurface model. The goal of our computation is to visualize how a wave travels through this model, displaying the velocity of the medium resulting from a pressure wave injected at the surface.

The remainder of this paper is structured as follows. Chapter 2 introduces the Marmousi2 model, the elastodynamic wave equation and its discretization, and the staggering and tiling techniques that motivate the design of both the sequential and parallel implementations. Chapter 3 describes how the sequential solver is structured and how the parallelization strategy using MPI and OpenMP is designed on top of it. Chapter 4 shows how this design is realized concretely in code for both the sequential and parallel versions. Chapter 5 presents performance measurements obtained on an HPC cluster, including strong and weak scaling behavior, the effect of tiling and output compression, and a trace-based analysis using Vampir. Finally, Chapter 6 interprets these results, identifies their limiting factors, and discusses directions for future work.

2 Background

2.1 Marmousi2 Model

Marmousi2[3] is an updated version of the original 1988 Marmousi[4] model. Its structure is based on the North Quenguela Trough in the Quanza Basin of Angola, a region composed mostly of shale with interbedded sand layers, a marl anticline within a faulted zone, an evacuated salt layer, and hydrocarbon traps near its centre.

Compared to the original model, Marmousi2 extends both the width and depth of the domain, increasing the width from 9.2 km to 17 km and adding 41 additional horizons, for a total of 199. The model was also made fully elastic through the addition of a shear-wave (S-wave) velocity field, alongside the existing pressure-wave (P-wave) velocity and density fields. A 450-meter water layer was also added at the top to represent a deep-water setting.

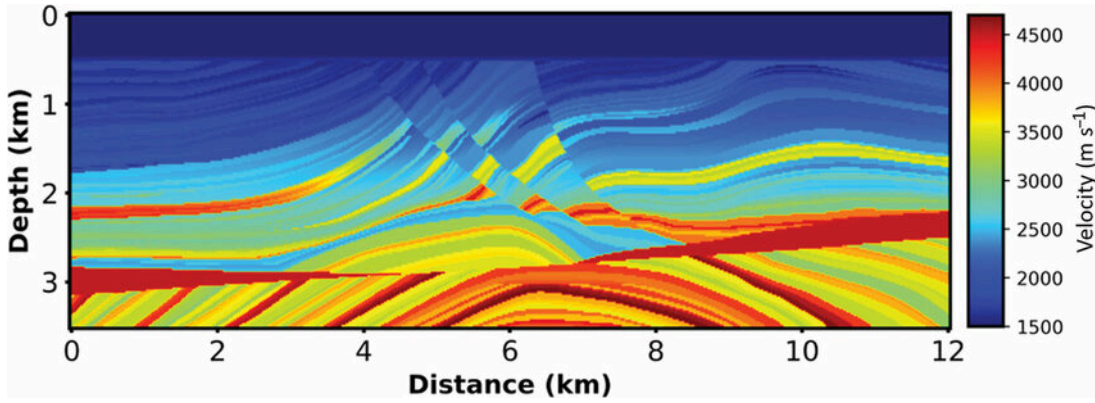


Figure 1: Marmousi2 Model

We chose it for three reasons. It is a standard benchmark, it is elastic and therefore allows for more realistic simulations, and at full resolution it is big enough that a parallel run is worth while.

The SEG open data collection [5] distributes the dataset as three SEG-Y files each holding v_p , v_s and ρ on a 1.25 m grid, which `segio` reads directly.

Quantity	Value
Physical grid $n_{z_0} \times n_{x_0}$	2801×13601
Physical grid points	38 095 201
Grid spacing $\Delta x = \Delta z$	1.25 m
Model extent (horizontal $\times$ depth)	17.0 km $\times$ 3.5 km

Table 1: Properties of the Marmousi2 model

2.2 Elastodynamic wave equation

Ultimately, the data computed by the simulation represents the particle velocity at every point in the material at every timestep, which is used to visualize how the wave travels through the medium. Since the source and the receivers of interest are placed near the surface, the waves of interest primarily travel upward, so the velocity component of interest is the one aligned

with that direction, the vertical velocity v_z . This also mirrors real seismic acquisition, where a receiver placed on the surface predominantly measures the vertical component of ground motion from an upward-arriving wave.

The velocity field is obtained by numerically solving the elastodynamic wave equation [6]. Several classes of methods exist for doing so, trading off complexity against accuracy. In this work, a finite difference method is used which approximates each derivative in the governing equations using the difference between nearby values on a grid.

The Marmousi2 model provides the P-wave velocity v_p , the S-wave velocity v_s , and the mass density ρ at every point of a realistic, geologically structured subsurface model. From these three quantities, we ultimately want to compute the velocity components v_x and v_z and the stress components σ_{xx} , σ_{zz} , and σ_{xz} at every grid point and every timestep. To do so, the elastic update equations additionally require the Lamé parameters λ and μ , which are not provided directly by the model but can be derived from v_p , v_s like this:

$$\begin{aligned}\mu &= \rho v_s^2 \\ \lambda &= \rho v_p^2 - 2\mu\end{aligned}$$

At each timestep, the stress and velocity variables are updated according to the following equations, which depend on the values at the previous timestep:

$$\begin{aligned}\frac{\partial v_x}{\partial t} &= \frac{1}{\rho} \left(\frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xz}}{\partial z} \right) \\ \frac{\partial v_z}{\partial t} &= \frac{1}{\rho} \left(\frac{\partial \sigma_{xz}}{\partial x} + \frac{\partial \sigma_{zz}}{\partial z} \right) \\ \frac{\partial \sigma_{xx}}{\partial t} &= (\lambda + 2\mu) \frac{\partial v_x}{\partial x} + \lambda \frac{\partial v_z}{\partial z} \\ \frac{\partial \sigma_{zz}}{\partial t} &= \lambda \frac{\partial v_x}{\partial x} + (\lambda + 2\mu) \frac{\partial v_z}{\partial z} \\ \frac{\partial \sigma_{xz}}{\partial t} &= \mu \left(\frac{\partial v_x}{\partial z} + \frac{\partial v_z}{\partial x} \right)\end{aligned}$$

2.3 Ricker Wavelet

To initiate wave propagation, a source term is added directly into the stress update at a single grid point near the surface, representing an idealized pressure disturbance analogous to an airgun or explosive source. The time dependence of this disturbance is given by a Ricker wavelet [7], a zero-mean pulse commonly used in seismic modeling to approximate the waveform produced by an impulsive source, defined as

$$r(t) = (1 - 2a)e^{-a}, \quad a = (\pi f_0(t - t_0))^2$$

where f_0 is the dominant frequency of the source, a fixed constant in this simulation, and t_0 delays the peak of the wavelet, computed from f_0 alone using a standard convention. At every timestep, the value of $r(t)$ is added directly into σ_{xx} and σ_{zz} at the source location, injecting an isotropic pressure disturbance that subsequently propagates outward through the model according to the update equations above.

2.4 Numerical Stability and the CFL Condition

Because the update equations are explicit, each new value depends only on values already known at the current timestep, the scheme is only stable if the timestep is chosen appropriately relative to the grid spacing and the wave speeds present in the model. This requirement is known as the Courant-Friedrichs-Lewy, or CFL, condition [8], and it follows directly from the combined timestep-to-grid-spacing factor introduced above. If the timestep were too large relative to the grid spacing, a wave could effectively advance by more than one grid cell within a single timestep, which the finite-difference stencil has no way of representing correctly, causing errors to grow instead of remaining bounded as the simulation progresses.

Since the P-wave is always the fastest wave present in an elastic medium, the stability limit is governed by the largest P-wave velocity found anywhere in the model, $v_{p,\max}$. Likewise, if the grid spacing differs between the two axes, the stricter of the two constraints comes from whichever spacing is smaller. The timestep used in this work is therefore chosen as

$$dt = 0.4 \cdot \frac{\min(dx, dz)}{v_{p,\max}}$$

where the factor 0.4 provides a safety margin below the theoretical stability limit to remain robust to the additional approximations present in the scheme, such as the damping applied near the absorbing boundaries.

2.5 Absorbing Boundary Damping

Reflections from the edges of the grid are suppressed using a simple damping mask condition. This technique is known as a sponge boundary [9]. The model is surrounded by padded regions on all four sides, and within these regions a damping value $d(i, j) \in [0, 1]$ is precomputed for every grid point.

In the interior of the model, $d(i, j) = 1$, meaning no damping is applied. Within the padded regions, $d(i, j)$ decreases smoothly from 1 (at the edge closest to the real model) down toward 0 (at the outer edge of the grid). The bottom boundary uses a stronger damping profile than the top and sides, so values there approach 0 more quickly.

At every time step, after the stress and velocity fields are updated, each field value is simply multiplied by the damping value at that location:

$$u(i, j) \leftarrow u(i, j) \cdot d(i, j)$$

for each of σ_{xx} , σ_{zz} , σ_{xz} , v_x , and v_z . Where $d = 1$, the value is unchanged. Where $d < 1$, the value is reduced slightly on that step. Because this multiplication happens at every time step, waves travelling through the padded region lose amplitude continuously, so that by the time they would reflect off the true edge of the grid and travel back into the model, their amplitude has been reduced to a negligible level.

2.6 Staggering

To increase the accuracy of the computation, staggering is applied. Staggering refers to deliberately storing or updating different quantities at offset positions, rather than at the same point, whether that offset is in space or in time. The two staggered quantities are never evaluated

at exactly the same location, but each is placed exactly halfway between two locations of the other. As shown below, this offset is what allows the finite differences used to update each field to reach second order accuracy without requiring a wider stencil.

2.6.1 Second order accuracy

Any time a continuous derivative is approximated using a finite difference, some error is introduced, since the approximation only uses a finite number of nearby values instead of the true, continuous function. The order of accuracy describes how quickly this error shrinks as the discretization is refined, that is, as the grid spacing Δ or timestep Δt is made smaller. A first order accurate scheme has an error that shrinks proportionally to Δ itself. Halving the spacing only halves the error. A second order accurate scheme has an error that shrinks proportionally to Δ^2 . Halving the spacing quarters the error. This means that for a small refinement, a second order scheme becomes accurate much faster than a first order one, since its error decreases quadratically. Without staggering, it would only grow linearly.

This distinction matters in practice because it directly affects how fine a grid or timestep is needed to reach a given accuracy target. A first order scheme generally requires a much finer discretization, and correspondingly far more computation, to reach the same accuracy as a second order scheme. Both the spatial and temporal staggering described below are specifically what allow the finite difference scheme used in this work to achieve second order accuracy.

2.6.2 Spatial Staggering

The finite difference scheme used to update the velocity and stress fields is applied on a spatially staggered grid, commonly referred to as a Virieux grid [10]. Rather than storing all five field components, v_x , v_z , σ_{xx} , σ_{zz} , and σ_{xz} , at the same physical location within a grid cell, each component is instead stored at a position offset by half a grid spacing relative to the others. Concretely, the normal stresses σ_{xx} and σ_{zz} are defined at integer grid points (i, j) , the horizontal velocity v_x is defined half a cell to the side at $(i, j + \frac{1}{2})$, the vertical velocity v_z is defined half a cell below at $(i + \frac{1}{2}, j)$, and the shear stress σ_{xz} is defined half a cell in both directions at $(i + \frac{1}{2}, j + \frac{1}{2})$.

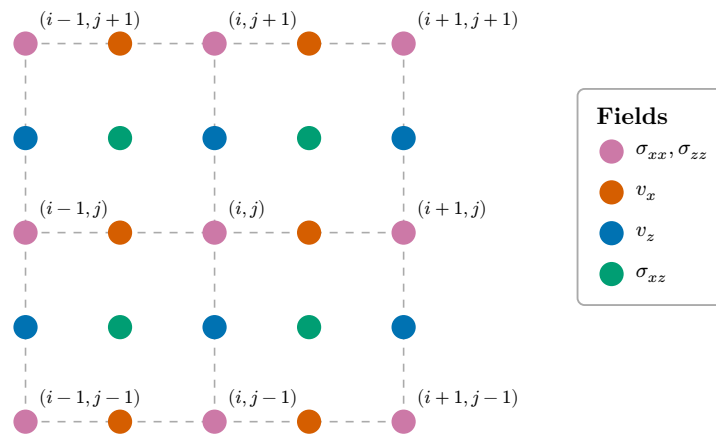


Figure 2: Layout of a portion of the staggered grid.

Placing the two fields on interleaved, offset grids means that whenever a derivative is needed, it is computed from the two nearest neighboring points on the opposite field's grid:

- v_x at $(i, j + \frac{1}{2})$: needs $\sigma_{xx}(i, j + 1)$, $\sigma_{xx}(i, j)$, $\sigma_{xz}(i + \frac{1}{2}, j + \frac{1}{2})$, $\sigma_{xz}(i - \frac{1}{2}, j + \frac{1}{2})$.
- v_z at $(i + \frac{1}{2}, j)$: needs $\sigma_{xz}(i + \frac{1}{2}, j + \frac{1}{2})$, $\sigma_{xz}(i + \frac{1}{2}, j - \frac{1}{2})$, $\sigma_{zz}(i + 1, j)$, $\sigma_{zz}(i, j)$.
- σ_{xx} and σ_{zz} at (i, j) : need $v_x(i, j + \frac{1}{2})$, $v_x(i, j - \frac{1}{2})$, $v_z(i + \frac{1}{2}, j)$, $v_z(i - \frac{1}{2}, j)$.
- σ_{xz} at $(i + \frac{1}{2}, j + \frac{1}{2})$: needs $v_x(i + 1, j + \frac{1}{2})$, $v_x(i, j + \frac{1}{2})$, $v_z(i + \frac{1}{2}, j + 1)$, $v_z(i + \frac{1}{2}, j)$.

2.6.3 Temporal Staggering

In addition to being staggered in space, the scheme is also staggered in time. Stress and velocity are not updated from a single shared snapshot of the fields. Instead, the two are updated at interleaved half time steps so that each update always consumes the most recently computed values of the other field. This is commonly known as a leapfrog scheme, and it makes the computation second order accurate in time.

2.7 Tiling

As a consequence of the temporal staggering described above, every timestep first advances the velocity fields by half a timestep using the stress values just computed, then advances the stress fields by the next half timestep, using the velocity values just computed. Implemented naively, this ordering can hurt performance. Since the entire velocity field is computed first, sweeping across the full grid, by the time this pass finishes, the values computed early in the sweep have long since been pushed out of the cache to make room for those computed later, as the cache cannot hold the full grid at once. When the stress update immediately afterward needs those same velocity values, most of them are no longer available nearby and must instead be streamed back in from main memory, a far slower operation than reading from cache, regardless of how many other cores happen to be active at the time. The resulting cost is especially significant relative to how cheap the underlying arithmetic is.

A second, separate effect appears once many cores are active simultaneously. Unlike the latency of a single memory access, which is a fixed cost paid independently by each core, the total bandwidth of the memory system is a shared resource. Every core on a socket draws from the same limited pool of throughput to main memory. As more threads are added, compute capacity grows with each additional core, but the bandwidth available to feed them does not, so cores increasingly compete for access to the same memory system. A computation whose performance is limited by either of these effects, the latency of individual accesses or the available shared bandwidth, rather than by how much arithmetic the processor can perform, is described as being memory bound.

This work applies tiling in an attempt to reduce both of these two effects. The grid is divided into many small tiles instead of computing the entire field in one pass. For each tile, stress is computed first, and immediately afterward, before moving on to another tile whose values would push these ones out of the cache, the corresponding velocity values are computed for that same tile while its stress values are still resident. Only once both fields have been fully advanced for one tile does the computation proceed to the next. In this way, the same round trip to main memory that would otherwise be required twice for every value, once to write it, once to read it back after it has been evicted, is reduced to a single round trip, since each value is consumed again while still cheap to access.

3 Methodology

3.1 Output

The primary output of the simulation is the vertical particle velocity v_z at every grid point, saved at regular intervals throughout the run. Each saved frame is visualized using Matplotlib, rendering v_z as a 2D image over the model domain. Since these frames are saved at fixed intervals, they can be assembled in sequence into a video, producing an animation of the wave as it propagates outward from the source and interacts with the structure of the Marmousi2 model.

3.2 Sequential Design

Every speedup in this report is measured against the sequential solver. It solves the same equations on the same grid with the same boundary treatment as the parallel version, and it writes the same output: the vertical particle velocity v_z , saved every hundredth step. Its inner loops are compiled C to make it a fair comparison with the parallel solver which needs to use C for this part of the program.

3.2.1 Structure of the program

In the following pseudo code, the solver is presented. Setup runs once: it reads the model, derives the arrays the kernels need, and prepares the output file. The time loop then repeats four operations for a certain number of times.

SETUP

```
read vp, vs, rho from three SEG-Y files
pad each field by nb cells
mu      <- rho * vs^2
lambda  <- rho * vp^2 - 2*mu
lam2mu  <- lambda + 2*mu
inv_rho <- 1 / rho
damp    <- quadratic sponge ramp on all four sides
dt      <- 0.4 * dx / max(vp)
```

TIME LOOP

```
for it = 0 .. n_iterations-1:
    inject_source(sxx, szz, vz, it, dt)
    update_stress_velocity(vx, vz, sxx, szz, sxz,
                          lam, lam2mu, mu, inv_rho, damp, dt)
    if it mod frame_stride == 0:
        save_frame(out, vz)
```

TEARDOWN

```
close the output file
```

```
function update_stress_velocity(vx, vz, sxx, szz, sxz, lam, lam2mu, mu, inv_rho,
damp, dt):
```

```
    for each tile (iz_tile, ix_tile) in domain:
        for i in iz_tile:
            for j in ix_tile:
                update sxx, szz, sxz
```

```

        apply damping
    for i in iz_tile:
        for j in ix_tile:
            update vx, vz
        apply damping

```

The four derived arrays, μ , λ , $\lambda + 2\mu$ and $1/\rho$, are computed once during setup. The kernels read exactly these quantities, so no material property is ever recomputed.

3.2.2 The Division Between Python and C

The program is written in two languages. Python reads the three model files, applies padding, derives the elastic parameters, builds the damping ramp, runs the time loop where it injects the pressure and calls the C kernels, and creates the output file. C handles the arithmetic, implemented as separate kernels compiled into a shared library and called from Python via `ctypes`. This division exists specifically to enable integration with OpenMP which is designed to be used in C programs.

3.2.3 Staggering

Spatial staggering is not achieved with offsets, there is no separate coordinate array marking a point as “ $i + \frac{1}{2}$ ”. Instead, v_x , v_z , σ_{xx} , σ_{zz} , and σ_{xz} are all stored as ordinary two dimensional arrays of the same shape, and the staggering exists only implicitly, in which neighboring array index each update kernel reads from. The offset shown in [Figure 2](#) is realized purely through the direction of the finite difference used at each point, not through any special indexing scheme.

Concretely, the stress update reads velocity one index ahead, a forward difference, since the velocity powering σ_{xx} conceptually sits half a cell beyond the current point: $\partial v_x / \partial x$ is approximated as the value of v_x one column to the right minus the value at the current point, divided by Δx , and $\partial v_z / \partial z$ is approximated the same way using the row directly below.

The velocity update reads stress one index behind instead, a backward difference, since the stress powering v_x conceptually sits half a cell before the current point: $\partial \sigma_{xx} / \partial x$ is approximated as the value at the current point minus the value one column to the left, and $\partial \sigma_{xz} / \partial z$ is approximated the same way using the row directly above.

Both updates perform an ordinary two point finite difference over grid points that are one full grid spacing apart, but because the stress update always looks forward and the velocity update always looks backward, the effective location each result represents is shifted by half a grid spacing relative to its input, without ever needing to track fractional indices.

Temporal Staggering is achieved by first updating stress and then velocity instead of both at once. While this is fused to accomplish tiling, this is still done sequentially.

3.3 Parallel Design

3.3.1 Parallelization Coverage

What was not parallelized

- **Time stepping loop.** The main loop advances the stress and velocity fields one iteration at a time, and each timestep depends on the result of the previous one. This dependency is the Amdahl bottleneck. Because of it the loop over time cannot be parallelized itself, only the work done within a single timestep can be.

- **One time setup.** Reading the SEGY model files, computing the Lamé parameters, building the PML damping profile and determining the domain decomposition all happen once before the time loop starts. This is currently repeated on every rank rather than distributed, since the cost is negligible compared to the roughly 50000 timesteps that follow. The final assembly of the per rank output files into one virtual HDF5[11] dataset also happens once, done serially by rank 0 after every rank has finished writing. Since this step only links metadata together using VirtualSource, it finishes almost instantly.
- **Pressure injection.** The source term is injected at a single grid point, so only the rank that owns this point performs the update. This step cannot be parallelized any further, and the resulting overhead is negligible.

What was parallelized

- **Domain decomposition.** The global grid is split across MPI[2] ranks using a 2D Cartesian topology built with `Compute_dims` and `Create_cart`. Each rank owns a rectangular subdomain plus a one cell halo. Neighboring ranks exchange halo values for the stress and velocity fields every timestep using `Sendrecv` calls.
- **Stress and velocity updates.** Inside each rank's subdomain, stress and velocity are updated using a fused, tiled kernel written in C and parallelized with OpenMP[1]. Combined with the MPI domain decomposition, this hybrid MPI and OpenMP scheme forms the main strategy of our parallelization.
- **HDF5 output.** Each rank writes its own wavefield snapshots to an independent file. This avoids the bottleneck that would occur if every rank had to send its data through rank 0 for writing.
- **Compression.** Blosc is used to compress the output data with OpenMP threads handling the compression work in parallel. Before this change, compression was performed by a single rank, meaning all other ranks were forced to wait idly until rank 0 finished compressing its share of the data before the program could proceed. Parallelizing compression with Blosc removed this serial bottleneck and led to a significant improvement in scaling performance.

3.3.2 MPI Parallelization

Domain Decomposition

The padded global grid of size $n_z \times n_x$ is distributed across MPI[2] ranks by arranging them on a two-dimensional Cartesian topology. Rather than relying on the default balancing provided by `MPI_Dims_create`, we search over all factor pairs (p_z, p_x) of the process count and choose the pair that minimizes $\frac{n_z}{p_z} + \frac{n_x}{p_x}$. Since the cost of the halo exchange scales with the perimeter of a subdomain, this avoids long, thin tiles that would otherwise increase communication relative to computation. Within each dimension, the grid is then split as evenly as possible, so that no rank is assigned a disproportionately large share of the domain and no rank becomes a bottleneck.

Once the Cartesian communicator is created, each rank locates its four neighbors automatically along the z and x axes. Ranks that lie on the boundary of the global domain receive a null neighbor on the corresponding side. Their halo exchange on that side is therefore skipped, and the PML absorbing layer is used instead of neighbor data. Figure 3 illustrates the resulting layout, with the PML region surrounding the inner grid of MPI ranks and one cell highlighted to show the halo shared between neighboring subdomains.

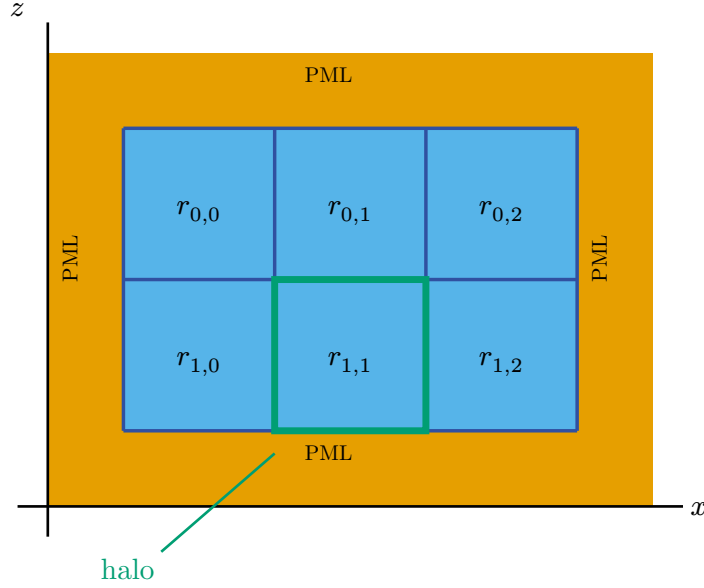


Figure 3: Decomposition of the padded grid into a Cartesian arrangement of MPI ranks, surrounded by the PML absorbing boundary. Each rank exchanges a one-cell-wide halo with its neighboring ranks every timestep.

Halo Exchange

As detailed earlier, the grid is staggered spatially, meaning that the velocity and stress components are not stored at the same physical location but are offset from one another by half a grid spacing. Physically, this means that a velocity value is treated as living midway between two neighboring stress points, rather than coinciding with them. Computing the spatial derivative needed to update it only requires the single stress value immediately behind it, not the values on both the left and the right. The same holds in reverse for updating stress from velocity. As a result, each rank only ever needs a neighboring value from one direction per axis instead of two, so only one side needs to be communicated for a given field instead of both sides. This cuts the data that has to be exchanged via MPI[2] in half. Without staggering all five fields would require neighbor values from both directions along each axis, giving ten directional transfers per axis, whereas with staggering the two velocity fields only require one direction and the three stress fields only require the other, giving five.

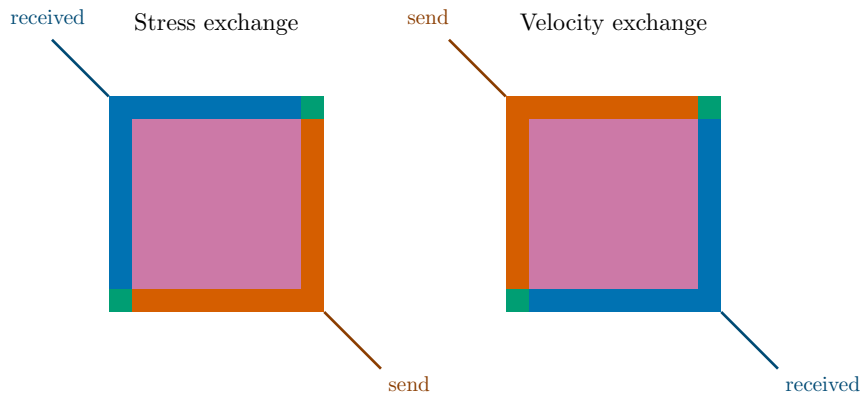


Figure 4: The two halo exchanges performed each timestep.

Execution Order and Tiling

Two separate exchanges are still required per timestep, one for the velocity fields and one for the stress fields which is a result of the temporal staggering. The stress fields must be fully updated using the exchanged velocity values before they can, in turn, be exchanged and used to update velocity.

The naive version previously applied performed each timestep in two fully separate sweeps. First, velocity was exchanged and used to compute stress everywhere, then stress was exchanged and used to compute velocity everywhere. This ordering is required by the temporal staggering described earlier, and it appears to prevent fusing the two kernels together, since velocity generally cannot be computed until the full stress exchange has completed.

But this is only true at the border of a rank's subdomain. Since the stencils used here reach only a single neighboring point, the vast majority of a rank's points depend only on data the rank already owns, regardless of any exchange. Only a thin strip at the edge actually needs data from a neighbor. This makes it possible to fuse stress and velocity for the interior immediately, while only the border falls back to waiting for the exchange, and since the interior does not depend on the exchange at all, the exchange can be issued in the background and left to complete while the interior is tiled, hiding its latency behind useful computation.

This latency hiding only applies to the stress exchange. The velocity exchange still has to complete beforehand, since the interior stress computation itself depends on it.

The velocity values at the upper left edge and the stress values at the lower right edge each have a dedicated kernel. At the rightmost column and lowest row, stress has already been computed before the fused kernel runs, in order to send it ahead of the backward halo exchange, so the fused kernel only needs to compute velocity for these points. At the top row and leftmost column, by contrast, the stress values needed for velocity are not yet available when the fused kernel runs, so the fused kernel only computes stress for these points, leaving their velocity to be completed afterward.

Concretely, one timestep is carried out in the following six steps, illustrated in [Figure 5](#):

1. `exchange_forward_halos`: velocity is exchanged with the minus-side neighbors, blocking until complete.
2. `update_stress_edges_c`: stress is computed for the thin border strip, using the freshly exchanged velocity.
3. `begin_backward_halos`: the border stress values just computed are sent to the plus-side neighbors, using a non-blocking call that returns immediately.
4. `update_stress_velocity_interior_c`: stress and velocity are fused and tiled for the interior, running while the exchange from step 3 completes in the background. Stress values at the upper left edge are calculated and velocity values for the lower right edge.
5. `finish_backward_halos`: the rank waits for the exchange from step 3 to complete and unpacks the received stress values.
6. `update_velocity_boundary_c`: velocity is computed for the border strip, using the stress values that just arrived.

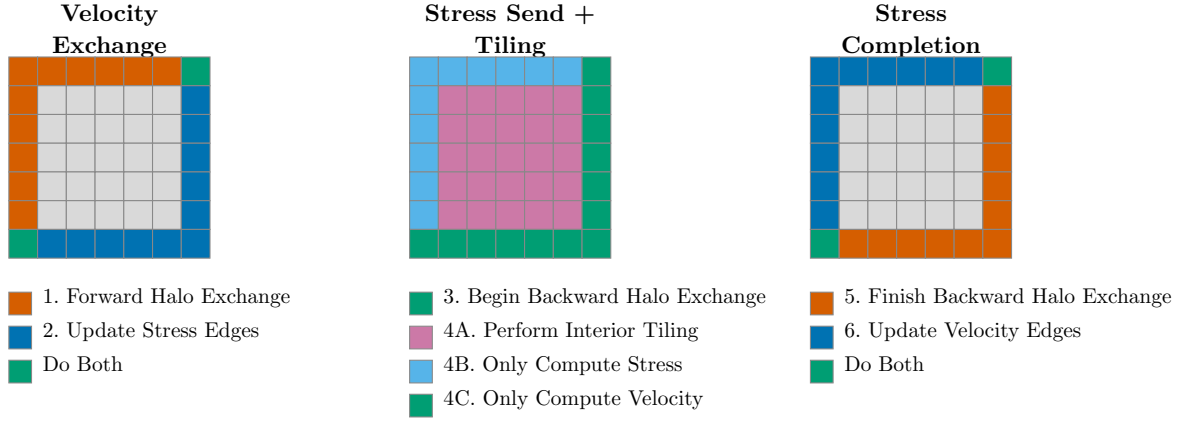


Figure 5: The three stages of one timestep, shown for a single rank's local subdomain. Cells that are involved in both are colored in two colors.

It is also worth noting that both halo exchanges cannot be hidden behind tiling, since the two exchanges must occur sequentially within a single timestep, while the fused kernel is only run once.

3.4 OpenMP Parallelization

While the previous sections describe how work is distributed across MPI ranks and how tiling reduces the memory traffic within a rank, this section describes how the work within a single rank is further split across the CPU cores available to it using OpenMP. The strategy presented here exploits two distinct, nested levels of parallelism, corresponding to the two axes of the local grid.

3.4.1 Tile Division

A tile is a portion of the grid for which stress is fully updated first, followed immediately by velocity, before moving on to the next tile in sequence. The purpose of this ordering, as established earlier, is that the newly computed stress values are still resident in cache when they are needed again moments later for the velocity update.

Since each CPU core has its own private L2 cache, a tile is not processed as a single unit but is itself divided into smaller row groups, one per OpenMP thread, so that every thread's share of the tile fits within its own core's cache rather than competing for space in a cache shared across cores. Within a tile, every thread first computes stress for its assigned rows, and only once every thread has finished, enforced by an implicit barrier, does any thread proceed to compute velocity for those same rows, ensuring the required stress values are both correct and still cache resident when they are read again.

How many rows each thread is assigned is therefore not arbitrary, but should be chosen so that the resulting working set, the row width multiplied by the number of fields stored per point and the size of each value, fits somewhat within the L2 cache. Once every thread has advanced through both the stress and velocity update for its rows within a tile, the computation proceeds to the next tile in the same fashion, continuing until the entire local grid has been processed.

To accomplish this, `#pragma omp for` is what actually assigns each thread its own row group within a tile, so that different threads work on entirely different rows at the same time. This

applies the parallelization along z . Figure 6 visualizes how this strategy is applied to a sub-grid using 2 OpenMP threads. In real runs, 16 threads were commonly used.

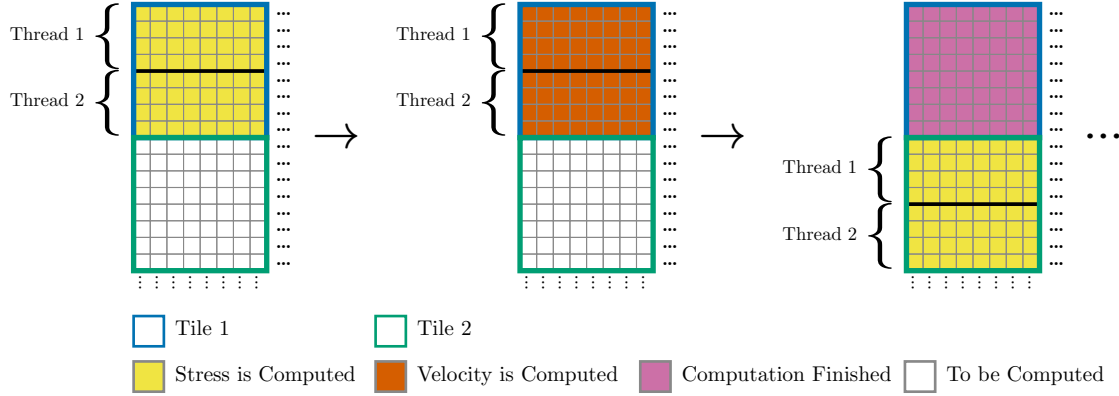


Figure 6: Processing tiles with OpenMP Using 2 Threads.

3.4.2 Vectorization with SIMD

Along x , the innermost loop over a single row is marked with `#pragma omp simd`, allowing the compiler to vectorize the update of many neighboring points using a single SIMD instruction on one core. This is independent of the parallelization along z . SIMD parallelism only speeds up the work each individual thread performs on its own assigned rows.

3.4.3 Handling the Domain Boundary When Tiling

The very first row of a rank's interior range, as well as the very leftmost column of every row, are treated specially. Their velocity depends on stress from the row directly above or the column directly to the left, both of which belong to the thin edge strip received from a neighboring rank since it is not available locally. The interior kernel therefore computes only stress for these points and skips their velocity entirely, leaving it to be computed later by the boundary kernel, once the corresponding halo data has arrived.

The bottom row and rightmost column are handled differently as well. Their stress is already computed early, by the edge kernel, specifically so that it can be sent to the neighboring rank without delay, and since that same stress value is exactly what their own velocity update needs, velocity for the bottom row and rightmost column is computed within the interior kernel.

3.4.4 Parallelizing the Edge Kernels

As was outlined in the MPI section, the stress values of the rightmost column and the bottom row are computed separately, and so are the velocity values of the leftmost column and the top row. These are parallelized with `#pragma omp for simd` for the single full row shared between both points (the bottom row for stress, the top row for velocity), combining thread-level and SIMD parallelism even for this thin strip, and with a plain `#pragma omp for` for the remaining column, which is split across threads point by point instead of being vectorized, since its access pattern is too irregular to benefit from SIMD. This ensures that every stage of the timestep, not just the interior, makes use of the available cores, even though the potential speedup is naturally much smaller here given how little work these strips represent relative to the interior.

4 Implementation

4.1 Setup

Hardware. All experiments were run on nodes equipped with Intel Xeon Platinum 8468 (“Sapphire Rapids”) processors, with 48 cores per socket and two sockets per node, giving 96 cores per node in total.

Grid and Model. Unless stated otherwise, results use the full resolution Marmousi2 grid of 2801×13601 points (Table 1), padded by 240 cells on each side for the absorbing boundary.

Timesteps. The simulation was run for 50000 timesteps.

Output. Every 100th frame is being saved giving 500 frames. Results are compressed using lz4.

4.2 Sequential

Throughout this section, $iz0$, $iz1$, $jx0$, $jx1$ denote the row and column bounds of the interior range being updated, and $last_i = iz1 - 1$, $last_j = jx1 - 1$ denote its final row and column. Since the arrays are stored as flat, row-major buffers, a point at row i and column j sits at flat index $k = i * nx + j$, where nx is the number of columns in the local grid, so $k + 1/k - 1$ reach the neighboring column and $k + nx/k - nx$ reach the neighboring row. This notation is reused unchanged in the Parallel section below.

4.2.1 From Continuous Derivatives to Array Differences

Every term of the form $\partial f / \partial t$ in the governing equations becomes a plain update rule once discretized: $f_{\text{new}} = f_{\text{old}} + dt \cdot (\text{right-hand side})$. Since dt/dx and dt/dz appear repeatedly, these two ratios are computed once per kernel call:

```
const float dtx = dt / dx;
const float dtz = dt / dz;
```

Each spatial derivative $\partial f / \partial x$ or $\partial f / \partial z$ is approximated by exactly two neighboring array reads.

4.2.2 The Stress Kernel

`update_stress` implements the three constitutive equations,

$$\begin{aligned}\frac{\partial \sigma_{xx}}{\partial t} &= (\lambda + 2\mu) \frac{\partial v_x}{\partial x} + \lambda \frac{\partial v_z}{\partial z} \\ \frac{\partial \sigma_{zz}}{\partial t} &= \lambda \frac{\partial v_x}{\partial x} + (\lambda + 2\mu) \frac{\partial v_z}{\partial z} \\ \frac{\partial \sigma_{xz}}{\partial t} &= \mu \left(\frac{\partial v_x}{\partial z} + \frac{\partial v_z}{\partial x} \right)\end{aligned}$$

Since the velocity components sit half a grid spacing ahead of the stress components on the staggered grid, both derivatives use a forward difference, reading the current point and its neighbor one step ahead:

```

const float dvx_dx = vx[k + 1] - vx[k];
const float dvx_dz = vx[k + nx] - vx[k];
const float dvz_dx = vz[k + 1] - vz[k];
const float dvz_dz = vz[k + nx] - vz[k];

```

Each of the three equations is then assembled term for term, using the precomputed Lamé combinations λ , $\lambda + 2\mu$, and μ read directly from the `lam`, `lam2mu`, and `mu` arrays:

```

const float sxx_new = sxx[k] + lam2mu[k] * dtx * dvx_dx + lam[k] * dtz * dvz_dz;
const float szz_new = szz[k] + lam[k] * dtx * dvx_dx + lam2mu[k] * dtz * dvz_dz;
const float sxz_new = sxz[k] + mu[k] * (dtz * dvx_dz + dtx * dvz_dx);

```

The absorbing boundary treatment described earlier is applied immediately afterward, by multiplying each updated value by the precomputed damping coefficient at that point before it is written back:

```

const float d = damp[k];
sxx[k] = sxx_new * d;
szz[k] = szz_new * d;
sxz[k] = sxz_new * d;

```

4.2.3 The Velocity Kernel

`update_velocity` implements the two momentum equations,

$$\frac{\partial v_x}{\partial t} = \frac{1}{\rho} \left(\frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xz}}{\partial z} \right)$$

$$\frac{\partial v_z}{\partial t} = \frac{1}{\rho} \left(\frac{\partial \sigma_{xz}}{\partial x} + \frac{\partial \sigma_{zz}}{\partial z} \right)$$

Here the roles are reversed: since stress sits half a grid spacing behind velocity on the staggered grid, both derivatives use a backward difference, reading the current point and its neighbor one step behind:

```

const float dsxx_dx = sxx[k] - sxx[k - 1];
const float dsxz_dz = sxz[k] - sxz[k - nx];
const float dsxz_dx = sxz[k] - sxz[k - 1];
const float dszz_dz = szz[k] - szz[k - nx];

```

The factor $1/\rho$ is read directly from the precomputed `inv_rho` array which is cheaper than dividing at every point, and the two equations are assembled and damped in the same way as the stress kernel:

```

const float ir = inv_rho[k];
const float vx_new = vx[k] + ir * (dtx * dsxx_dx + dtz * dsxz_dz);
const float vz_new = vz[k] + ir * (dtx * dsxz_dx + dtz * dszz_dz);
const float d = damp[k];
vx[k] = vx_new * d;
vz[k] = vz_new * d;

```

4.2.4 Tiling the Loop Nest

Both kernels sweep the same local grid. The grid is divided into small tiles, applying the tiling strategy motivated in the Background section. The outer two loops step through the grid in blocks of `tile_z` rows and `tile_x` columns, and only once a tile has been fully advanced does the computation move on to the next one:

```

for (int ib = iz0; ib < iz1; ib += tile_z) {
    int ie = min(ib + tile_z, iz1);
    for (int jb = jx0; jb < jx1; jb += tile_x) {
        int je = min(jb + tile_x, jx1);
        for (int i = ib; i < ie; ++i) {
            for (int j = jb; j < je; ++j) {
                ...
            }
        }
    }
}

```

To make comparisons fair, the tiling is applied here too, although it was meant as an optimization strategy for parallelization. It still provides cache optimization improvements. `tile_z` and `tile_x` are chosen so that the working set of the ten arrays involved, five fields and five material properties, for one tile remains small enough to stay resident in the core's own cache while both the stress and velocity update for that tile are computed. This is the same tiling technique used in the parallel implementation, applied here without any OpenMP directives, since a single core has no work to divide among threads.

4.3 Parallel

4.3.1 MPI

The decomposition search, halo exchange, and six-step execution order described in the Parallel Design section are realized directly in the driver. This section shows how each of those pieces is written in code.

Building the Cartesian Communicator

The (pz, px) pair chosen by the decomposition search is passed to `Create_cart`, from which each rank derives its coordinates and its four neighbors along both axes:

```

cart = comm.Create_cart(dims=dims, periods=[False, False], reorder=False)
coord_z, coord_x = cart.Get_coords(rank)
z_minus, z_plus = cart.Shift(0, 1)
x_minus, x_plus = cart.Shift(1, 1)

```

Forward Exchange

The forward exchange sends each rank's first real row and column to its minus-side neighbors and receives into the plus-side ghost cells, corresponding to step 1:

```

def exchange_forward_halos(fields):
    for k, field in enumerate(fields):
        send_x[k] = field[1:-1, 1]
        cart.Sendrecv(send_x, dest=x_minus, recvbuf=recv_x, source=x_plus)
    if x_plus != MPI.PROC_NULL:
        for k, field in enumerate(fields):
            field[1:-1, -1] = recv_x[k]
    # repeated for the z axis using field[1, 1:-1] and field[-1, 1:-1]

```

This is called once, on [vx, vz], before any computation happens in a timestep.

Backward Exchange

The backward exchange is split into two calls corresponding to steps 3 and 5, so that the interior can be computed in between. `begin_backward_halos` packs the last real row and column and issues non-blocking sends and receives:

```
def begin_backward_halos(fields):
    for k, field in enumerate(fields):
        send_x[k] = field[1:-1, -2]
    requests = [
        cart.Irecv(recv_x, source=x_minus, tag=30),
        cart.Isend(send_x, dest=x_plus, tag=30),
    ]
    # repeated for the z axis using field[-2, 1:-1]
    return requests
```

`finish_backward_halos` is called after the interior kernel has run, and only then blocks on the transfer before unpacking the received values into the minus-side ghost cells:

```
def finish_backward_halos(fields, requests):
    MPI.Request.Waitall(requests)
    if x_minus != MPI.PROC_NULL:
        for k, field in enumerate(fields):
            field[1:-1, 0] = recv_x[k]
    # repeated for the z axis
```

Every kernel call in between passes NumPy arrays directly through `ctypes`, with no copying and no MPI call inside any kernel, so communication and computation remain fully separate at the code level.

4.3.2 OpenMP

Tile Size

The tile size described in the Parallel Design section is computed once per run by `get_tile_rows`, defaulting to four rows per thread, clamped to between 16 and 256, and overridable through an environment variable for tuning:

```
int rows = 4 * omp_get_max_threads();
if (rows < 16) rows = 16;
if (rows > 256) rows = 256;
```

The Tiled Loop

`update_stress_velocity_interior` opens a single `#pragma omp parallel` region around the entire tile loop, so the thread team is created once per kernel. Each iteration of the outer loop processes one tile, `ib` to `ie`, first computing stress for every row in the tile, then, in a second `#pragma omp for`, computing velocity for the same rows:

```
#pragma omp parallel
{
    for (int ib = iz0; ib < last_i; ib += tile_rows) {
        int ie = min(ib + tile_rows, last_i);
        #pragma omp for schedule(static)
        for (int i = ib; i < ie; ++i) {
            stress_row(vx, vz, sxx, szz, sxz, lam, lam2mu, mu, damp,
                       dtx, dtz, nx, i, jx0, last_j);
        }
        #pragma omp for schedule(static)
        for (int i = max(ib, iz0 + 1); i < ie; ++i) {
```

```

        velocity_row(vx, vz, sxx, szz, sxz, inv_rho, damp,
                    dtx, dtz, nx, i, jx0 + 1, jx1);
    }
}

```

`schedule(static)` divides `[ib, ie)` into contiguous, equally sized blocks decided once for the loop, so a thread is assigned the same rows in the velocity loop that it just computed stress for in the loop above. Since both loops sit inside the same `#pragma omp parallel` region, the implicit barrier at the end of the first `#pragma omp for` is what enforces the ordering: every thread has finished writing stress for the entire tile before any thread starts reading it in the second loop, without any explicit synchronization being written. The velocity loop starts one row later than the stress loop (`max(ib, iz0 + 1)`) and one column later (`jx0 + 1`), since the very first row and column of the interior range depend on halo data and are deferred to the boundary kernel, as described above. After the tile loop finishes, one further `#pragma omp for simd` pass computes velocity for the interior's last row, whose stress was already available locally.

Inside a Row

`stress_row` and `velocity_row` receive the row index `i` and the column bounds from the loop above and compute the flat row offset once before iterating over columns:

```

static inline void stress_row(..., int i, int jx0, int jx1) {
    const int row = i * nx;
    const int rowp = row + nx;
    #pragma omp simd
    for (int j = jx0; j < jx1; ++j) {
        const int k = row + j;
        ...
    }
}

```

By the time this function runs, `#pragma omp for` has already assigned row `i` to exactly one thread, so `#pragma omp simd` here only vectorizes across `j`, the columns of that single row.

Edge and Boundary Kernels

`update_stress_edges` and `update_velocity_boundary` each open their own, shorter `#pragma omp parallel` region. In both, the single full row, which is the bottom row for stress and the top row for velocity, is parallelized with `#pragma omp for simd`, combining both levels directly on one loop:

```

#pragma omp for simd schedule(static)
for (int j = jx0; j < jx1; ++j) {
    const int k = last_i * nx + j;
    ...
}

```

The remaining column, one point per row, is handled by a separate point-wise helper (`stress_point`, `velocity_point`) and split across threads with a plain `#pragma omp for`, without vectorization, since a single point per iteration gives the compiler nothing to vectorize:

```

#pragma omp for schedule(static)
for (int i = iz0; i < last_i; ++i) {
    stress_point(vx, vz, sxx, szz, sxz, lam, lam2mu, mu, damp,
                dtx, dtz, nx, i, last_j);
}

```

5 Results

5.1 Wave Propagation Data

The program successfully produced the data required to animate the seismic wave forward modeling. Figures [Figure 7](#), [Figure 8](#) and [Figure 9](#) display three select frames taken from the final animation.

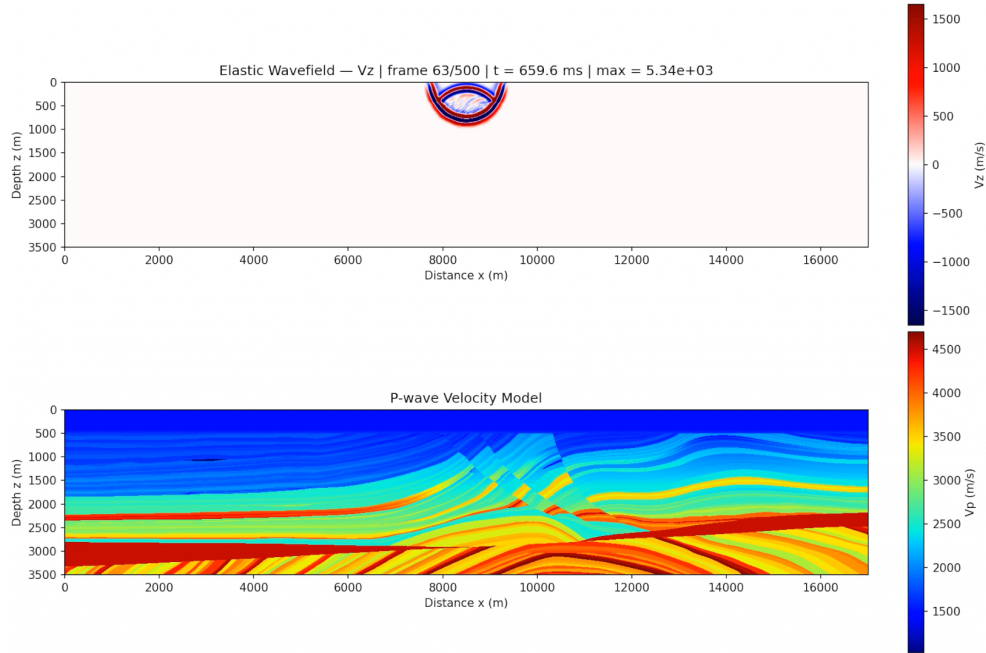


Figure 7: Output Frame 63

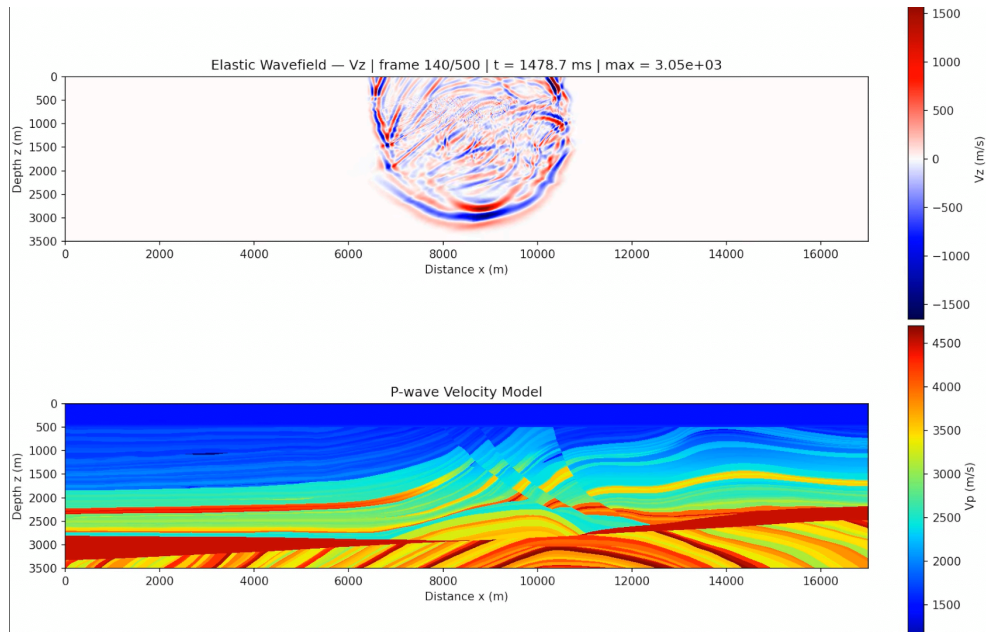


Figure 8: Output Frame 140

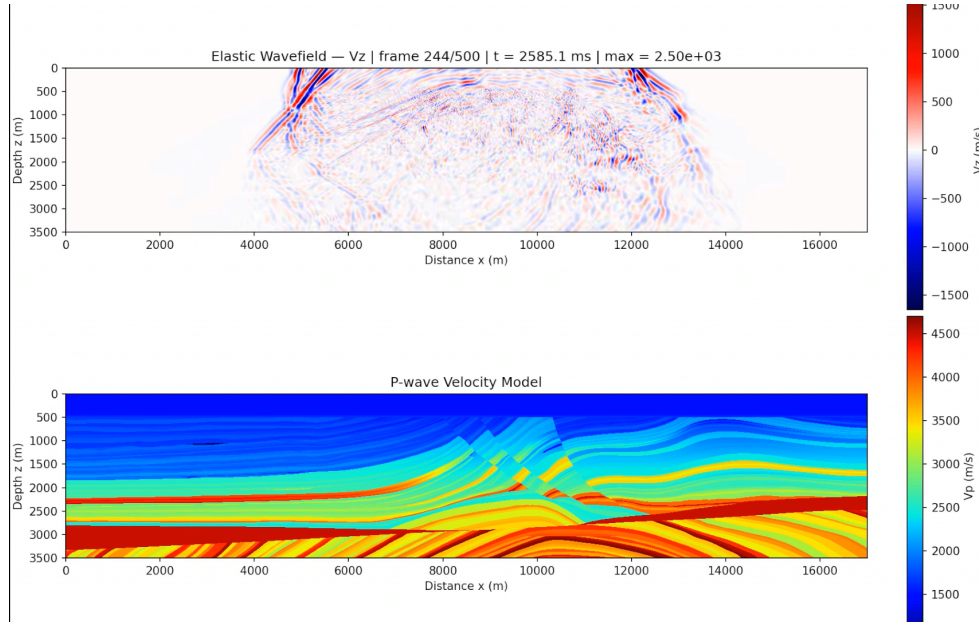


Figure 9: Output Frame 244

5.2 Sequential Performance

5.2.1 Overall Runtime

The average of the overall runtime of four computations was 12374 seconds, or 3h 26m 14s. This reflects 50000 iterations over the padded compute grid of 3281 x 14081 points, roughly 46.2 million points per iteration once the 240 cell absorbing boundary on each side is included. This total includes setup, the full time loop, and output writing, and is broken down further in the next section. This value also serves as the baseline runtime used for all speedup and efficiency calculations in the Parallel Performance section below.

5.2.2 Runtime Breakdown by Program Phase

To analyze how execution time is distributed across the program, `perf`[\[12\]](#) was used to generate a flame graph, shown in [Figure 10](#).



Figure 10: A flame graph showing the relative time spent in each part of the sequential solver.

The flame graph shows that setup, compression, and teardown are all negligible compared to the main simulation loop, which dominates total runtime. This is why the parallelization effort for the final solver used in the benchmarks below targets the main loop specifically. However, as parallelization accelerates the main loop, the relative cost of compression grows more significant, and compression performance was optimized as well for that reason.

5.3 Parallel Performance

The parallel performance of the implementation was evaluated using strong- and weak-scaling experiments. Strong scaling measures how effectively additional compute resources reduce the runtime of a fixed problem. Weak scaling instead measures how well the runtime is maintained when the problem size and the available resources are increased together. Each result is the average of at least four computations.

Each configuration is written as $n \times m \times s$, where n is the number of nodes, m is the number of MPI ranks per node, and s is the number of OpenMP threads assigned to each rank. The total number of CPU cores used by configuration i is therefore

$$P_i = n_i \times m_i \times s_i. \quad 1.$$

For example, the configuration $2 \times 6 \times 16$ uses two nodes, six MPI ranks per node, and 16 threads per rank, for a total of 192 CPU cores. The configuration $1 \times 1 \times 1$ denotes the purely sequential execution, where a separate program was used that employs no OpenMP or MPI at all, as opposed to the same parallel program launched with one rank and one thread.

Each reported runtime is the arithmetic mean of four independent runs. Let T_i denote the mean runtime of configuration i , let P_i denote its total number of CPU cores as defined in [Equation 1](#), and let configuration 0 denote the smallest-resource configuration in the corresponding experiment.

5.3.1 Strong Scaling

In a strong-scaling experiment, the global problem size remains constant while the number of CPU cores is increased. The speedup relative to the baseline is

$$S_{i,0} = \frac{T_0}{T_i}.$$

Ideal scaling would reduce the runtime in direct proportion to the increase in resources. The strong-scaling efficiency relative to the baseline is therefore

$$E_{i,0} = \frac{S_{i,0}}{\frac{P_i}{P_0}} = \frac{T_0}{T_i} \times \frac{P_0}{P_i}.$$

The incremental efficiency relative to the preceding configuration is

$$E_{i,i-1} = \frac{\frac{T_{i-1}}{T_i}}{\frac{P_i}{P_{i-1}}} = \frac{T_{i-1}}{T_i} \times \frac{P_{i-1}}{P_i}.$$

An efficiency of 1.0 represents ideal linear scaling. A value below 1.0 means that the achieved speedup is smaller than the increase in resources, while a value above 1.0 represents super-linear scaling. Efficiency relative to the baseline describes the accumulated scaling behavior over the complete resource range, while incremental efficiency describes the effect of a single transition. The incremental efficiency of the first configuration is undefined because no preceding configuration exists.

Strong Scaling on a Single Node

[Figure 11](#) shows the strong-scaling results for a fixed problem executed with different CPU core counts on a single node, using a single MPI rank with an increasing number of OpenMP threads.

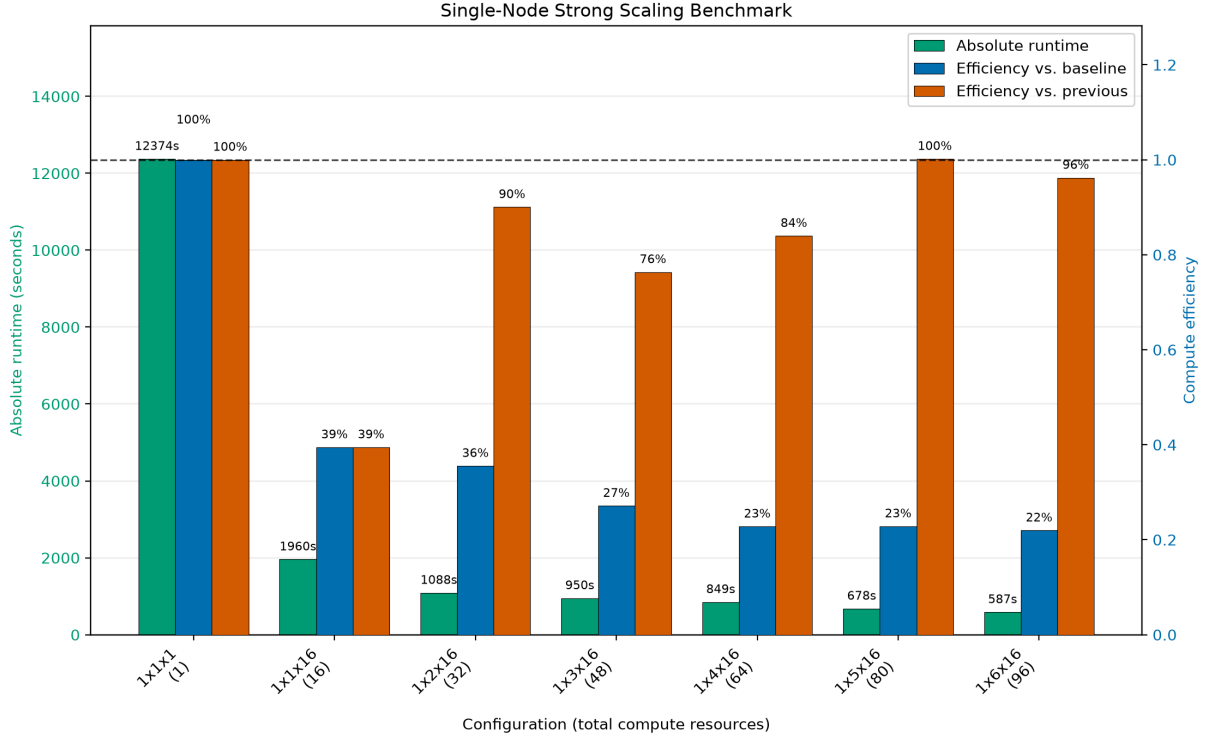


Figure 11: Strong-scaling results on a single node for one MPI rank and an increasing number of OpenMP threads.

The biggest efficiency drop occurs between the sequential solver and the 1x1x16 run, where efficiency falls to just 39%. Some of this can be explained by the overhead introduced by OpenMP itself, including thread synchronization barriers and general parallelization overhead that a sequential run does not incur. However, we suspect that memory latency and bandwidth already play a major role even at this core count. This is further supported by the Vampir [13] trace analysis below, which suggests that a large fraction of the computation is still spent waiting for data to arrive from memory rather than computing. In the subsequent runs, efficiency relative to the previous core count consistently stays around 85%, a reasonable rate of diminishing returns. So while efficiency at 96 cores drops to only 22%, this is only about half of the 39% efficiency already lost at 16 cores, rather than a further collapse of similar size. Ultimately, parallelizing this computation on a single CPU using 96 cores brought the runtime down from 12374 seconds to just 587 seconds.

Strong Scaling Across Multiple Nodes

Figure 12 shows the strong-scaling results across multiple nodes. Each node runs six MPI ranks with 16 threads per rank, for a total of 96 CPU cores per node.

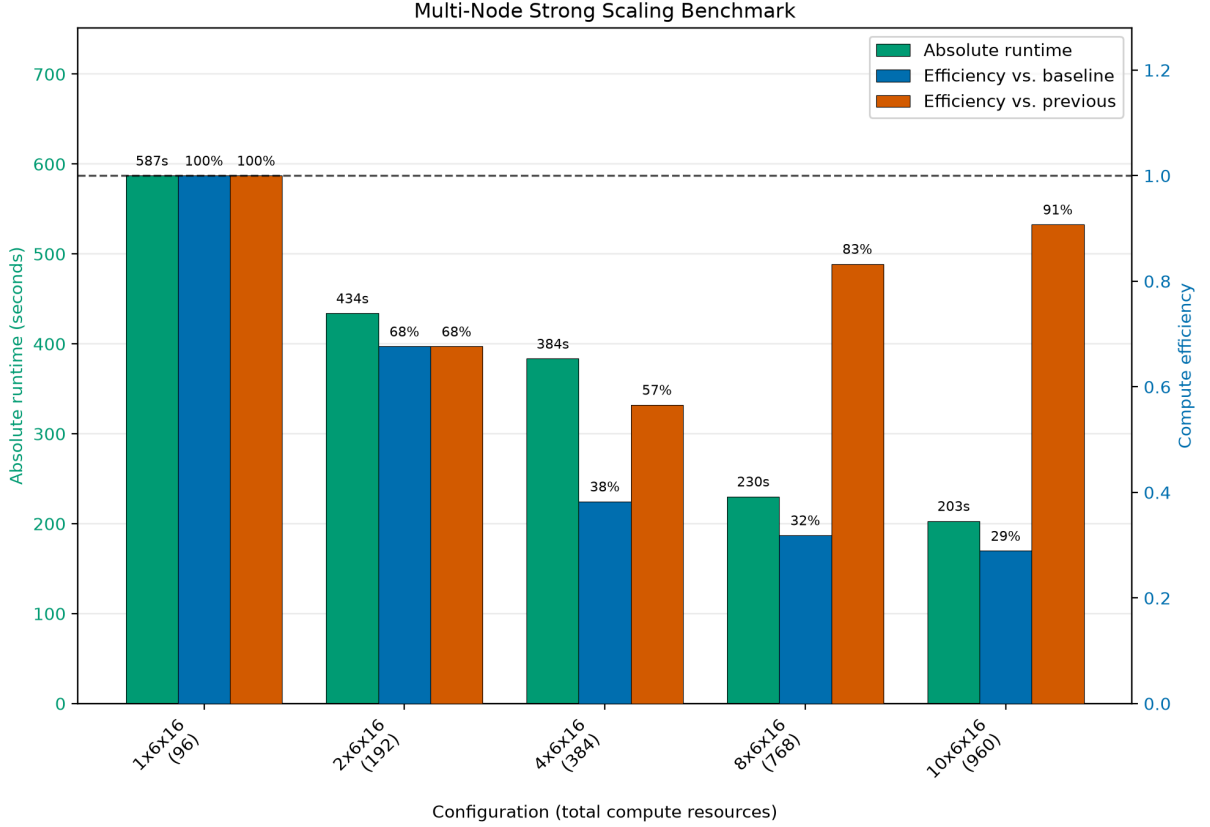


Figure 12: Strong-scaling results across multiple nodes with six MPI ranks per node and 16 threads per rank.

Using multiple nodes, the computation could be sped up further, from 587 to 203 seconds, corresponding to an efficiency of 29%. What stands out is that while incremental efficiency drops for the two and four node runs, it rises again strongly for the 8 and 10 node runs. It is unclear why this is the case. It cannot, however, be attributed to noise, since the same pattern also appears in the weak scaling results.

5.3.2 Weak Scaling

In a weak-scaling experiment, the global problem size is increased approximately in proportion to the number of CPU cores, so that the workload assigned to each core remains approximately constant. The method used to adjust the spatial dimensions is described in the implementation section.

Strong-scaling efficiency is not applicable in this case, because the configurations solve problems of different sizes. Weak scaling is instead evaluated using relative runtime. The runtime relative to the first configuration is

$$R_{i,0} = \frac{T_i}{T_0},$$

and the runtime relative to the preceding configuration is

$$R_{i,i-1} = \frac{T_i}{T_{i-1}}.$$

A relative runtime of $1.0 \times$ represents ideal weak scaling. A value of $1.2 \times$ means that the execution took 20 percent longer than the reference execution, while a value of $0.8 \times$ means that it required only 80 percent of the reference runtime. The term *relative runtime* is used rather than *slowdown*, since values below $1.0 \times$ represent a reduction in runtime. The comparison with the preceding configuration is undefined for the first configuration.

Weak Scaling on a Single Node

Figure 13 shows the weak-scaling results obtained on a single node, using a single MPI rank with an increasing number of OpenMP threads.

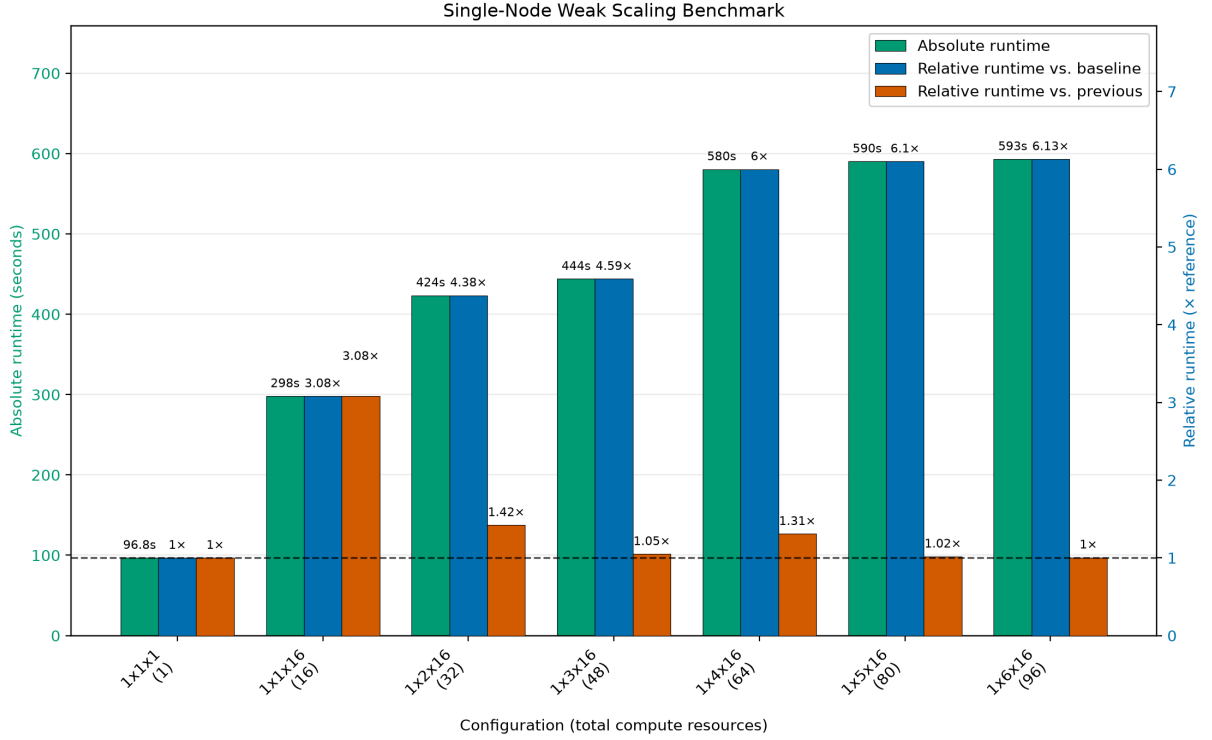


Figure 13: Weak-scaling results on a single node.

The weak-scaling result is broadly consistent with its strong-scaling counterpart. A relative runtime of $6.13 \times$ corresponds to a weak-scaling efficiency of $\frac{1}{6.13} \approx 16.31\%$, which is lower than the approximately 22% efficiency observed in the strong-scaling experiment. The cause of this difference is unclear. The fact that the runtime stays consistent towards higher core counts confirms the high incremental efficiency observed in the strong scaling results.

Weak Scaling Across Multiple Nodes

Figure 14 shows the corresponding weak-scaling results across multiple nodes, using six MPI ranks per node and 16 threads per rank.

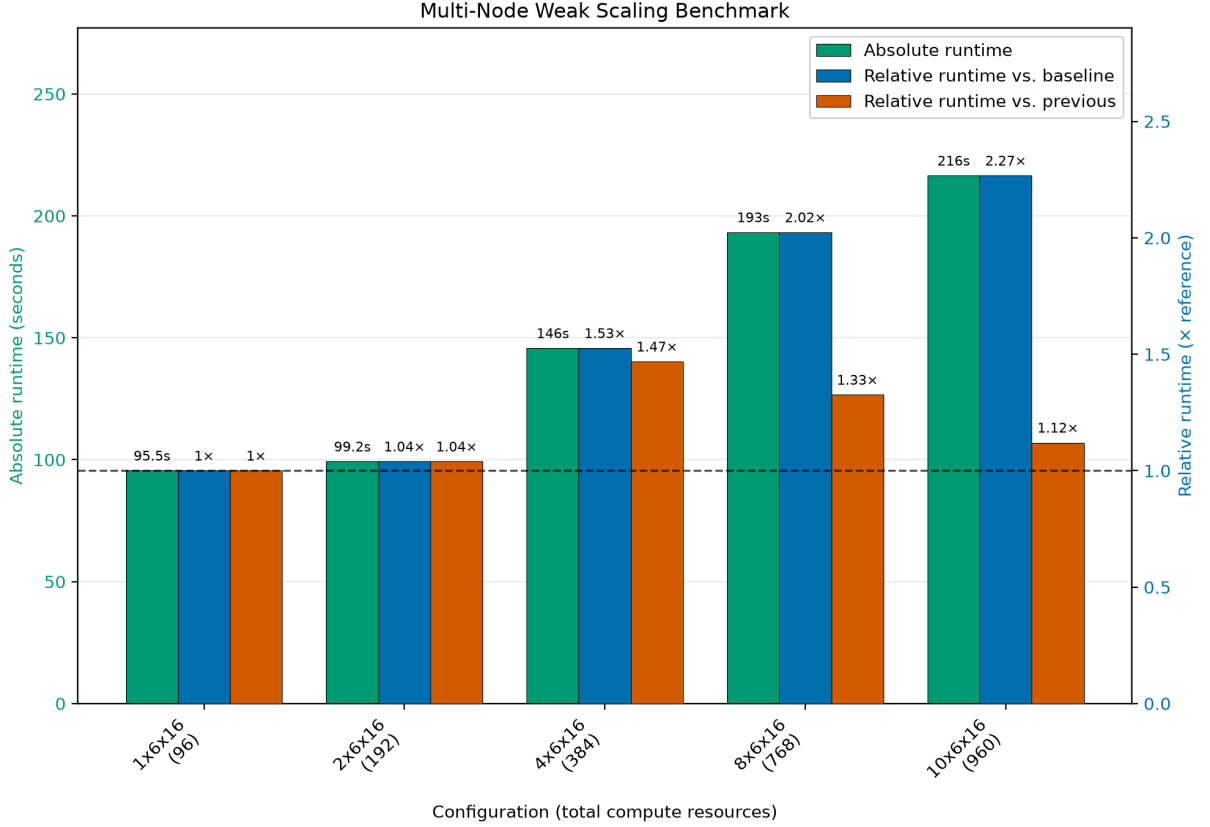


Figure 14: Weak-scaling results across multiple nodes with six MPI ranks per node and 16 threads per rank.

The multi-node weak-scaling result is broadly comparable to its strong-scaling counterpart. A relative runtime of $2.27 \times$ corresponds to a weak-scaling efficiency of $\frac{1}{2.27} \approx 44.05\%$, which is higher than the approximately 29% efficiency observed in the strong-scaling experiment. Importantly, the relative-runtime ratio decreases again for the two configurations after 4x6x16. This trend is consistent with the increase in incremental efficiency observed for these same configurations relative to the preceding, lower-node-count configurations in the strong-scaling benchmark. Both benchmarks therefore indicate that scaling improves again at the higher node counts. It is unclear to us why the weak scaling is performing significantly better.

5.3.3 MPI Ranks and Threads per Rank

Figure 15 compares configurations that all use 96 CPU cores on a single node, distributed differently between MPI ranks and OpenMP threads per rank.

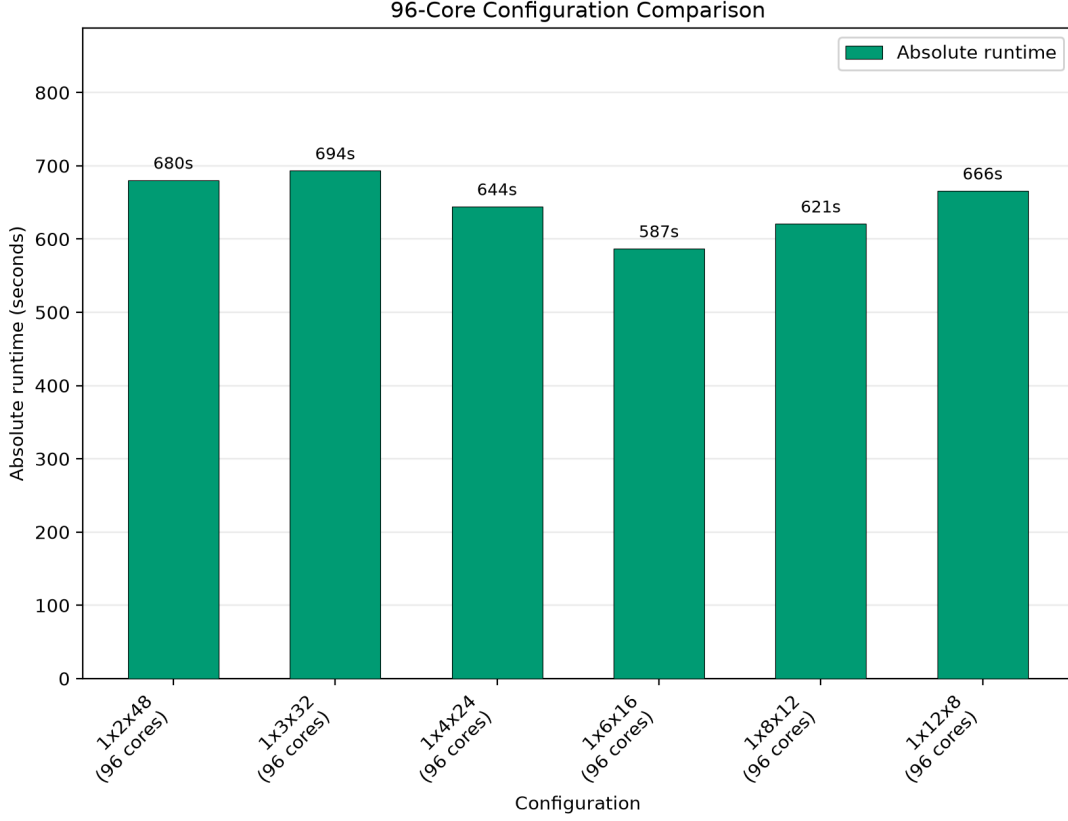


Figure 15: Mean runtime for different decompositions of 96 CPU cores into MPI ranks and OpenMP threads per rank.

The configuration used in the benchmarks was informed by this experiment, which shows that 16 OpenMP threads per MPI rank is the most efficient choice for most runs. This reflects a tradeoff between the two decomposition strategies. Increasing the number of OpenMP threads per MPI rank increases the latency introduced by thread synchronization barriers, since each barrier must wait for the slowest of the participating threads. Conversely, increasing the number of MPI ranks relative to the number of OpenMP threads increases the number of subdomains and therefore the amount of MPI halo exchange, which increases communication latency.

5.3.4 Non-Tiling Version

Tiling was used to improve performance specifically in higher core count per CPU runs. To demonstrate that impact, [Figure 16](#) compares the final version of the solver to an earlier iteration which did not use tiling.

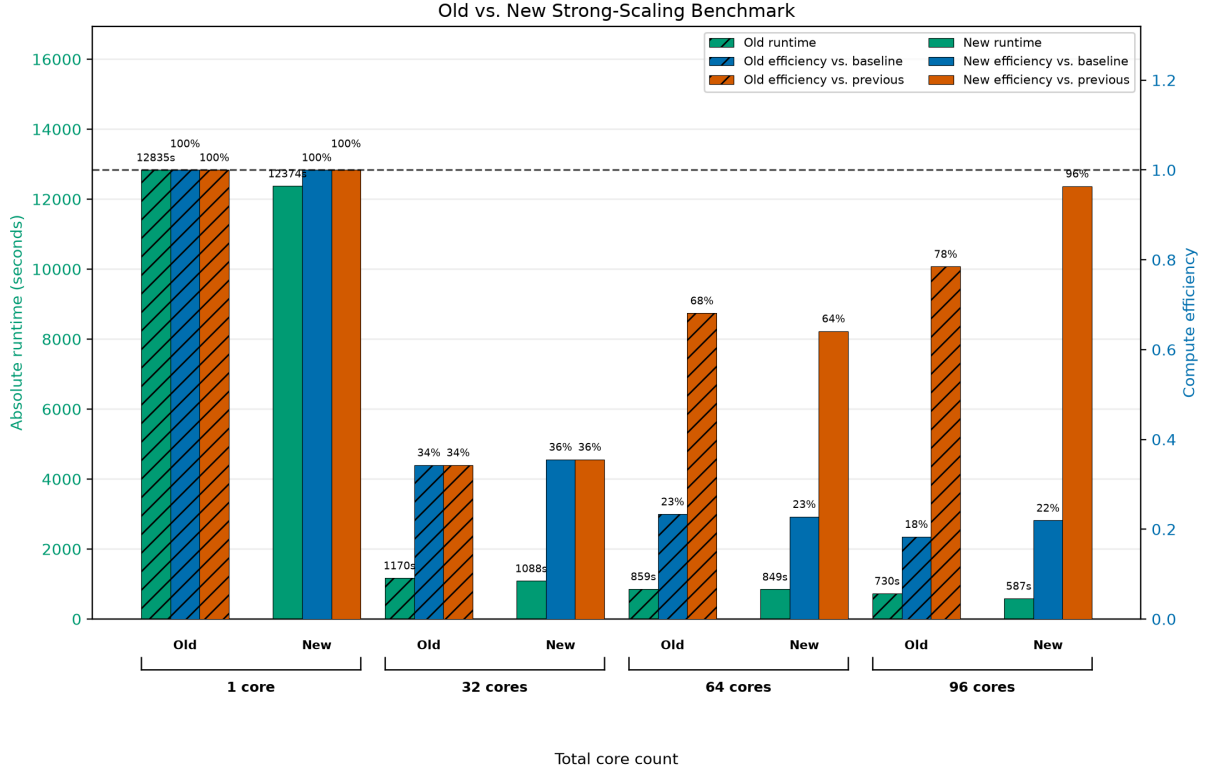


Figure 16: Performance of tiling vs no-tiling version. Only core counts are provided since the old version preferred a 32 core per rank configuration.

Tiling generally improved performance by making better use of the cache. Even the sequential solver benefited slightly, with runtime decreasing from 12835 to 12374 seconds, a 3.6% improvement. The effect was considerably larger for the 96 core runs, where runtime dropped from 730 to 587 seconds, a 19.6% improvement. This also raised parallel efficiency relative to the sequential version from 18% to 22%, a relative improvement of 22.2%.

There are two likely reasons the improvement was not larger. First, the original, untiled version already used fewer barriers, so tiling’s added synchronization overhead partially offsets its benefit. Second, memory latency, and possibly bandwidth, likely still persist as limiting factors, merely reduced rather than eliminated by tiling. That the improvement is more pronounced at higher core counts, however, suggests that tiling is indeed working as intended, since this is exactly where memory related bottlenecks would be expected to matter most.

5.3.5 Kernel Comparison

Multiple OpenMP kernels were evaluated to identify the best-performing implementation although the current version includes optimizations beyond the kernel-level as was described in the implementation section. Although several kernels outperformed the original implementation, the tiled implementation remained the fastest. The benchmark results are shown in [Figure 17](#).

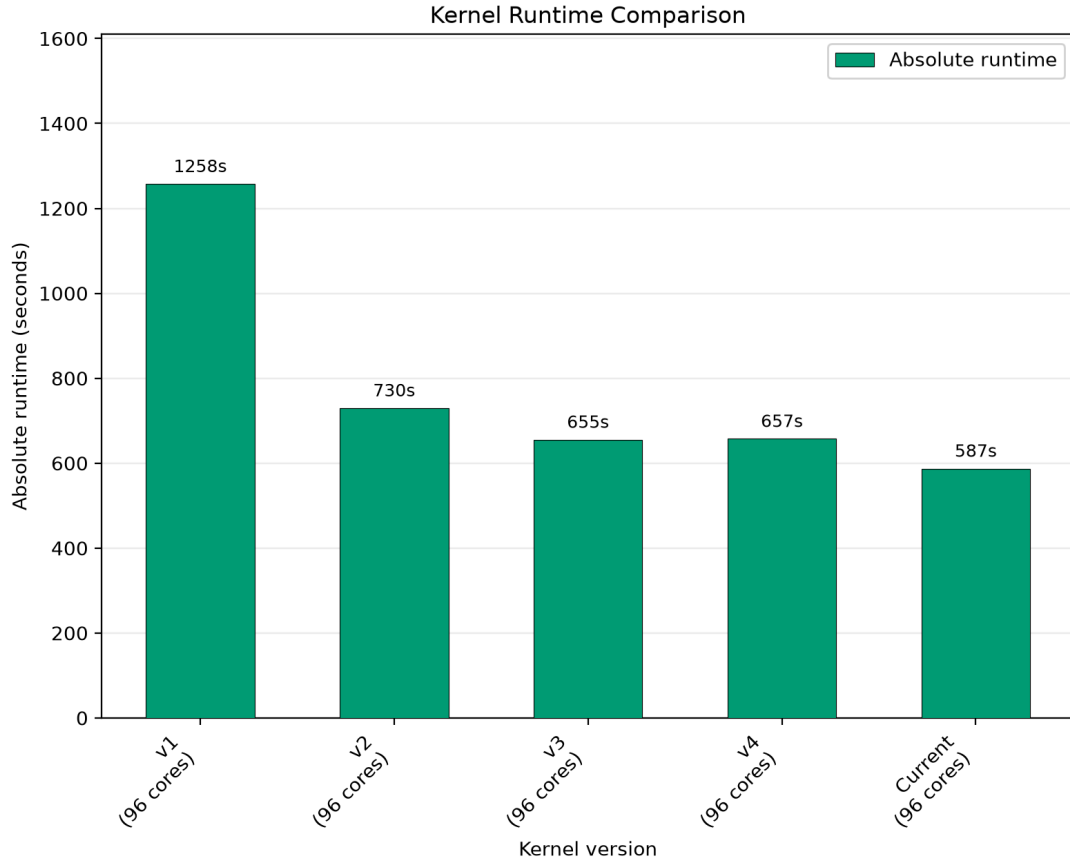


Figure 17: Performance relative to various OpenMP kernels.

5.3.6 Output Compression

As described earlier, the compression is parallelized as well to avoid the bottleneck that is created when 16 OpenMP threads are available while only one is used for compression. lz4 is used as the compression algorithm in the benchmarks. Further, this is also compared against no compression and compression using gzip. The results are shown in [Figure 18](#).

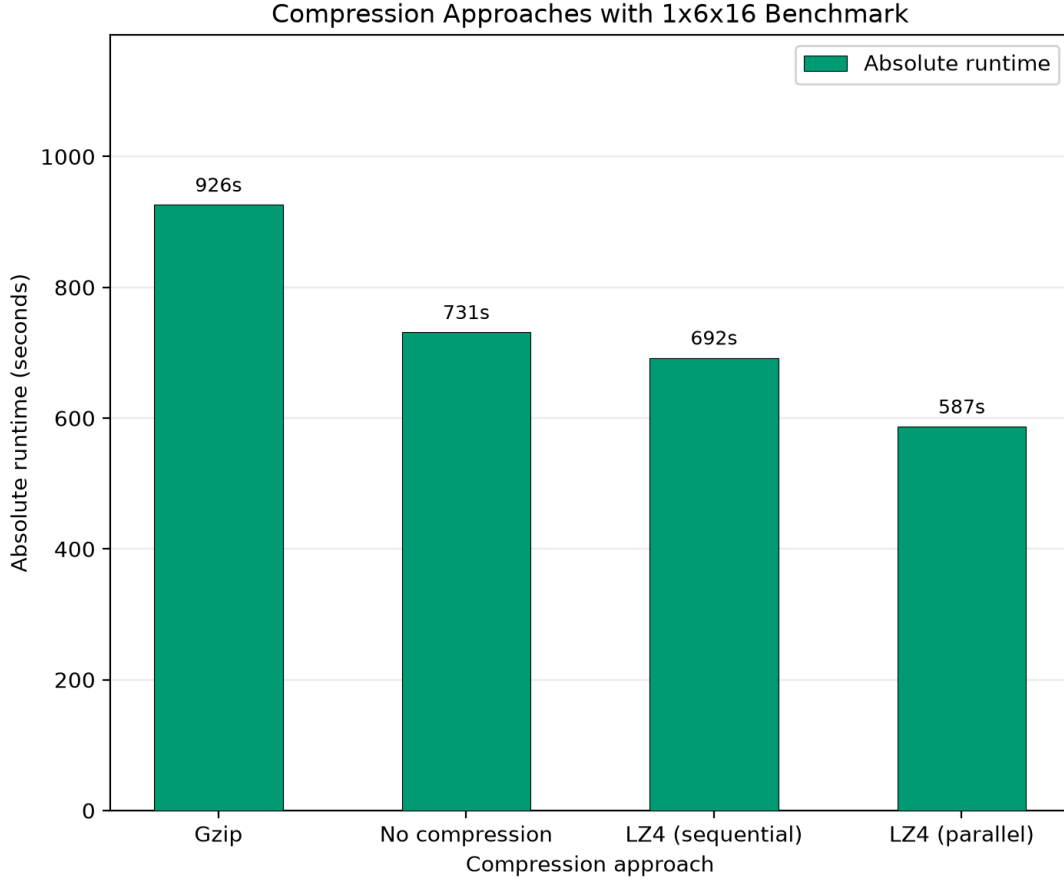


Figure 18: Effect of output compression on total runtime.

Gzip was the slowest with 926 seconds, followed by no compression with 731 seconds, then sequential LZ4 with 692 seconds, and finally parallel LZ4 with 587 seconds, which was the fastest. The gap between gzip and LZ4 is expected since LZ4 is known to be much faster. It is also unsurprising that parallelizing the LZ4 compression improved performance further compared to the sequential version.

What is surprising is that no compression was slower than even sequential LZ4. This contrasts with earlier runs using fewer ranks and threads where no compression outperformed compression. The likely explanation is that with more ranks and threads, compression becomes fast enough that the reduced I/O volume it produces (roughly 35GB instead of 70GB) saves more time than the compression itself costs. At this scale the time saved by writing half as much data to disk outweighs the time spent compressing it while at smaller scales the compression overhead dominated instead. It is therefore demonstrated that the benchmarks presented use the fastest compression method tested.

5.4 Trace Based Analysis with Vampir

5.4.1 Measurement Setup

To examine where time is spent at the level of individual functions and threads, the program was instrumented with Score-P 8.4 [14] and the resulting OTF2 traces were inspected in Vampir [13]. The C kernels were compiled with the Score-P compiler wrapper, which instruments every OpenMP construct through OPARI2. These regions appear in the trace named after their source location, for example `!$omp for @elastic_kernels_tiled.c:296`. MPI calls are recorded through

the Score-P MPI wrappers, and the Python driver runs under the Score-P Python bindings, so driver functions such as `exchange_forward_halos` and the h5py call `Dataset.__setitem__` appear in the trace as well.

5.4.2 OpenMP Barriers

Figure 19 visualizes the tiling behavior inside the C kernels for a run using 6 ranks with 16 threads each. Orange indicates time spent computing, while cyan indicates a thread waiting at an OpenMP barrier.

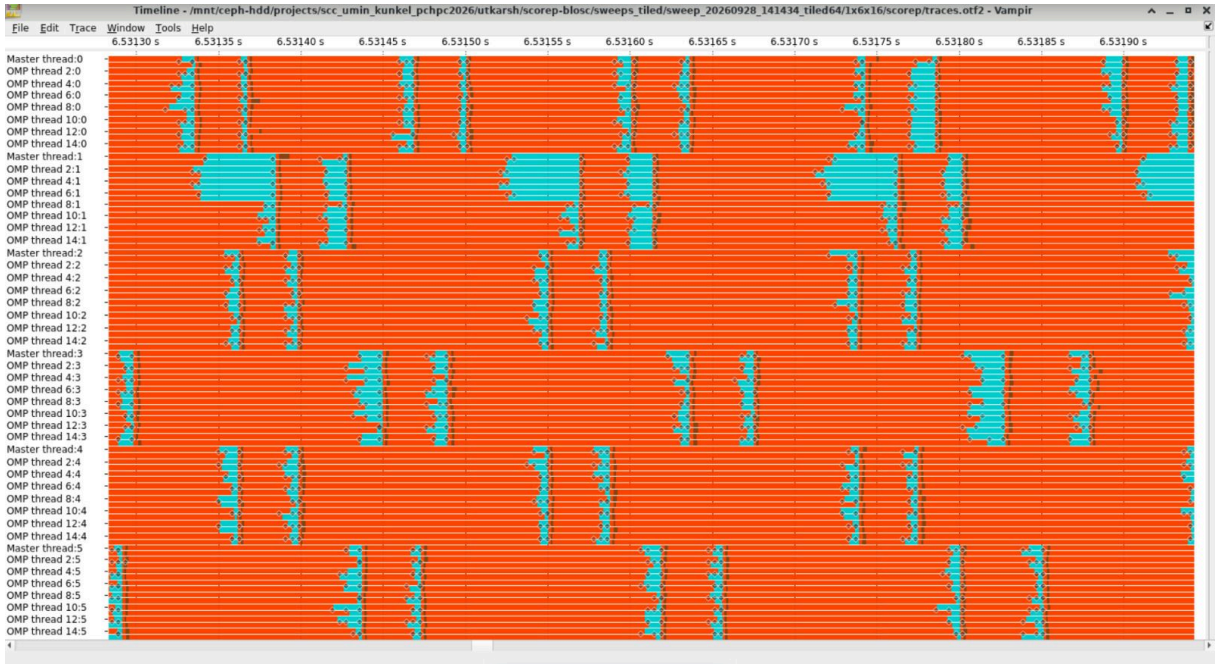
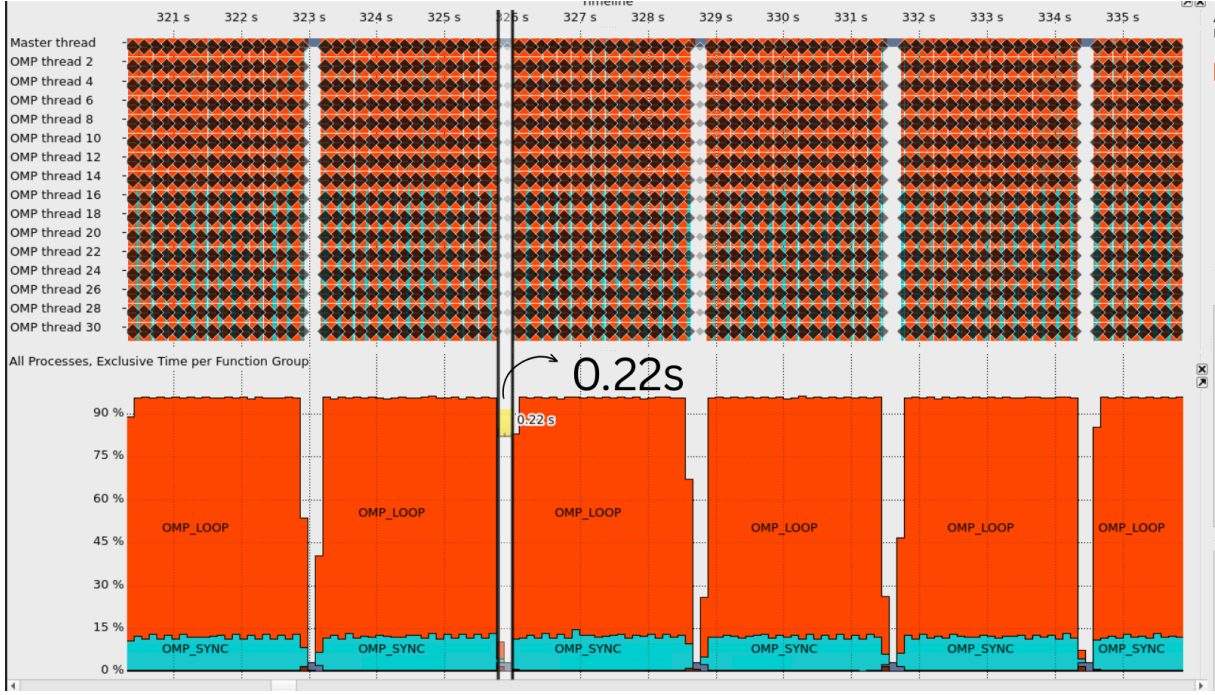


Figure 19: Tiling Kernel Visualization Using Vampir

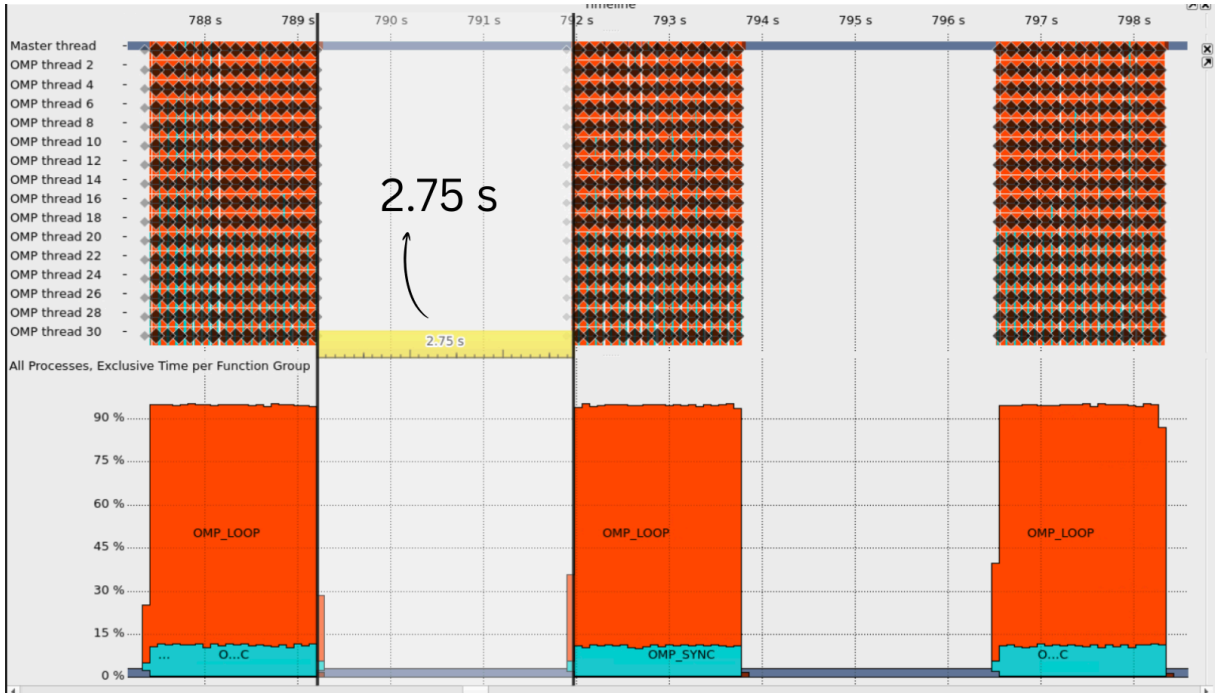
Each rank clearly operates on its own independent rhythm of long and short computation phases, separated by short cyan barrier segments. Within each rank, the long phase corresponds to the stress computation and the short phase to the velocity computation, matching the two separate `#pragma omp for` loops executed per tile. The most striking feature is just how large this difference is. The stress update does involve somewhat more arithmetic than the velocity update, but that alone comes nowhere close to explaining a gap of this size. The far more plausible explanation is the effect tiling was specifically designed to produce. Computing stress requires reading velocity values that were last written a full timestep, and many other tiles, ago, and are therefore long gone from cache, whereas computing velocity immediately afterward reads the stress values just written by the previous phase for the same tile, still sitting hot in cache and accessible with far less latency. This visualization confirms our memory bottleneck theory.

5.4.3 Gzip, LZ4 Comparison

Figure 20 compares traces of gzip and LZ4 (via Blosc). This was done on an older version of the code, since tiling could not be visualized without heavy downsampling as more Score-P data is produced by it, which would have reduced the amount of data being compressed and therefore would not have represented compression performance accurately. An older version was used instead, and since the amount of data saved per frame is the same, the results should be identical in the most recent version using tiling.



(a) Blosc LZ4



(b) gzip

Figure 20: Vampir visualization of blosc LZ4 vs gzip.

With Gzip compression, kernel execution pauses for approximately 2.75s, whereas with LZ4 using Blosc the pause is shorter, at approximately 0.22s. The Score-P trace did not capture the Blosc compression routines themselves, so their parallel execution could not be visualized.

6 Discussion

6.1 Analysis and Bottleneck

The sequential run, using the full resolution grid with no downsampling and 50000 iterations, took 12374 seconds. The fastest parallel run, using 10 nodes with 96 cores each (a 960 fold increase in compute resources), took 203 seconds. This corresponds to a speedup of 61 times, a significant improvement, but only a fraction of the 960 fold increase in resources, yielding a parallel efficiency of 6.4%.

It is clear that at this scale we are deep into diminishing returns. Using only 16 cores on a single node already reduced the runtime to 1960 seconds, a speedup of 6.3 times relative to the sequential run. This suggests that most of the achievable efficiency is captured early, while scaling to hundreds of cores yields comparatively small additional gains.

This scaling behavior reflects three main limiting factors, the OpenMP barriers introduced by tiling, the forward MPI halo exchange, which, unlike the backward exchange, could not be hidden behind interior computation, and memory latency, and possibly bandwidth, which tiling was able to reduce but not eliminate. Nevertheless, the earlier bottleneck related to compression was successfully addressed and the tiling did improve the memory issues somewhat although not as much as was hoped. However, overall the result is satisfactory, and the tiling mechanism demonstrates effectiveness in mitigating the memory bandwidth limitations, even if it could not remove them entirely.

6.2 Improvements

While the parallelization strategy presented in this work achieves substantial speedups over the sequential baseline, real world production use of seismic forward modeling is typically accelerated using GPUs rather than, or in addition to, multi core CPUs. The update kernels used here are a good example of a workload well suited to this. The computation performed at each grid point is simple and identical across the entire grid, with no data dependent branching, which maps well onto the thousands of lightweight threads a GPU provides. This could make it possible to extend this computation to 3D data as well, something that remains largely impractical with the current version.

Additionally, the memory bandwidth restriction remains unproven. LIKWID [15] was used to attempt to measure memory bandwidth usage, we used `likwid-perfctr` with the `MEM` performance group on the kernel. While core-level counters (instruction and cycle counts) were read correctly, the memory-controller counters required for bandwidth computation consistently returned zero. Verbose diagnostic output revealed that these counters were never actually queried by LIKWID, suggesting a permissions restriction. To demonstrate the memory bandwidth bottleneck of earlier versions and how well tiling solved it, this measurement would be highly informative.

And finally, the amount of work allocated per thread within a tile is fixed at 4 rows. This value has simply been hardcoded. The idea was to make the per-thread workload as large as possible to reduce the number of round trips to memory, while still keeping it small enough for all the stress values to fit into the cache and minimize memory access overhead. However, as

the subdomain size shrinks with an increasing number of ranks, the effective cost and benefit of using exactly 4 rows changes as well as the rows shorten. An algorithm could be designed to adjust the row count dynamically based on cache size and rank count in order to minimize the number of memory accesses. This is likely the most impactful improvement that could be explored in future work. Varying the row count per thread has not yet been benchmarked.

7 Conclusion

This work presented a hybrid MPI and OpenMP implementation of elastic seismic wave forward modeling on the Marmousi2 subsurface model at full resolution. The sequential solver was extended with a domain decomposition across MPI ranks, where the global grid was split into rectangular subdomains chosen to balance their shape and reduce the relative cost of halo exchange, with each rank exchanging halo values with its neighbors every timestep. OpenMP was used to split the work within each rank across threads and vectorize it with SIMD. Tiling, introduced as a cache optimization for the parallel implementation specifically, kept the stress and velocity updates operating on data that remained resident in cache within each rank, which likely improved performance by noticeably reducing, though not eliminating, the impact of memory bottlenecks. Output writing was similarly parallelized, with each rank writing its own compressed file using LZ4 rather than funneling data through a single rank, removing a serial bottleneck that had previously left most ranks waiting on compression performed by only one of them.

Using this approach, the runtime was reduced from 12374 seconds sequentially to 203 seconds using 960 cores across ten nodes, a speedup of 61 times. Compared to the diminishing returns visible already at smaller core counts, where 16 cores gave a speedup of 6.3 times, this result is reasonable given the combined overhead of OpenMP synchronization and MPI halo exchange at larger scale.

LIKWID was used in an attempt to directly measure memory bandwidth utilization, though the relevant counters could not be read due to what appears to be a permissions restriction, and this measurement could be attempted again in future work. Given the nature of the workload, with simple, uniform computation performed independently at every grid point, GPU acceleration is another natural direction for future work which could improve performance substantially. And the performance of the current solver could still be improved by adjusting the row count allocated per thread in the tiling algorithm to the rank count.

A Bibliography

- [1] L. Dagum and R. Menon, “OpenMP: an industry standard API for shared-memory programming,” *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, Jan. 1998, doi: [10.1109/99.660313](https://doi.org/10.1109/99.660313).
- [2] M. P. I. Forum, “MPI: A message - passing interface standard,” 1994. [Online]. Available: <https://api.semanticscholar.org/CorpusID:58052736>
- [3] G. Martin, R. Wiley, and K. Marfurt, “Marmousi2: An elastic upgrade for Marmousi,” *The Leading Edge*, vol. 25, pp. 156–166, Feb. 2006, doi: [10.1190/1.2172306](https://doi.org/10.1190/1.2172306).
- [4] R. Versteeg, “The Marmousi experience: Velocity model determination on a synthetic complex data set,” *The Leading Edge*, vol. 13, no. 9, pp. 927–936, 1994, doi: [10.1190/1.1437051](https://doi.org/10.1190/1.1437051).
- [5] “Open Data.” [Online]. Available: https://wiki.seg.org/wiki/Open_data
- [6] K. Aki and P. Richards, “Quantitative Seismology, 2nd Ed,” Jan. 2002.
- [7] N. Ricker, *The Form and Laws of Propagation of Seismic Wavelets*, vol. 3rd World Petroleum Congress. in World Petroleum Congress (WPC), vol. 3rd World Petroleum Congress. 1951, p. WPC–4039.
- [8] R. Courant, K. Friedrichs, and H. Lewy, “Über die partiellen Differenzgleichungen der mathematischen Physik,” *Mathematische Annalen*, vol. 100, no. 1, pp. 32–74, 1928, doi: [10.1007/BF01448839](https://doi.org/10.1007/BF01448839).
- [9] C. Cerjan, D. Kosloff, R. Kosloff, and M. Reshef, “A Nonreflecting boundary-condition for discrete acoustic and elastic wave-equations,” *Geophysics*, vol. 50, pp. 705–708, Apr. 1985, doi: [10.1190/1.1441945](https://doi.org/10.1190/1.1441945).
- [10] J. Virieux, “P-SV wave propagation in heterogeneous media: Velocity-stress finite-difference method,” *Geophysics*, vol. 51, pp. 889–901, Jan. 1984, doi: [10.1190/1.1442147](https://doi.org/10.1190/1.1442147).
- [11] M. Folk, G. Heber, Q. Koziol, E. Pourmal, and D. Robinson, *An overview of the HDF5 technology suite and its applications*. 2011, pp. 36–47. doi: [10.1145/1966895.1966900](https://doi.org/10.1145/1966895.1966900).
- [12] Linux Kernel Developers, “perf: Linux profiling with performance counters.” 2026.
- [13] A. Knüpfer, H. Brunst, J. Doleschal, and others, “The Vampir Performance Analysis Tool-Set,” in *Tools for High Performance Computing*, Springer, 2008, pp. 139–155. doi: [10.1007/978-3-540-68564-7_9](https://doi.org/10.1007/978-3-540-68564-7_9).
- [14] A. Knüpfer, C. Rössel, D. an Mey, and others, “Score-P: A Joint Performance Measurement Run-Time Infrastructure for Periscope, Scalasca, TAU, and Vampir,” in *Tools for High Performance Computing 2011*, Springer, 2012, pp. 79–91. doi: [10.1007/978-3-642-31476-6_7](https://doi.org/10.1007/978-3-642-31476-6_7).
- [15] J. Treibig, G. Hager, and G. Wellein, “LIKWID: A Lightweight Performance-Oriented Tool Suite for x86 Multicore Environments,” in *2010 39th International Conference on Parallel Processing Workshops*, 2010, pp. 207–216. doi: [10.1109/ICPPW.2010.38](https://doi.org/10.1109/ICPPW.2010.38).

B Work Sharing

B.1 Maxim Barnstorf

Maxim was responsible for the core implementation of the simulation, from the initial sequential solver through to the final parallel, tiled version, making heavy use of LLMs to assist with the underlying mathematics. The implementation included the main simulation loop, covering the ingestion of the input model files, construction of the absorbing boundary damping profile, and the design and implementation of the tiled update kernels used in the final version. He implemented the MPI domain decomposition and halo exchange, and integrated OpenMP based tiling into the C kernels to reduce memory traffic. He additionally resolved the output bottleneck by parallelizing the compression step and restructuring the HDF5 output so that each rank writes its own file independently. He also attempted to measure memory bandwidth utilization using LIKWID.

B.2 Utkarsh Pathak

Utkarsh developed the additional OpenMP kernel variants v1, v3 and v4 evaluated in the kernel comparison benchmark, and carried out the corresponding performance benchmarking. He contributed to an earlier iteration of the sequential solver, and was responsible for the trace based performance analysis, instrumenting the code with Score-P and producing the resulting Vampir visualizations used to investigate OpenMP barrier behavior and I/O. He also implemented the visualization and plotting of the wavefield output, and refactored the codebase and prepared it for submission which included basic documentation to use the program.